

HỌC VIỆN CÔNG NGHỆ BƯU CHÍNH VIỄN THÔNG
KHOA CÔNG NGHỆ THÔNG TIN 1
HỌC PHẦN PHÁT TRIỂN CÁC HỆ THỐNG THÔNG MINH



ASSIGNMENT 05

**ĐỀ TÀI: MẠNG NƠ-RON TÍCH CHẬP NHÌN NHƯ HỢP CỦA CÁC HÀM,
TỪ CNN CƠ BẢN ĐẾN CÁC KIẾN TRÚC PHÁT TRIỂN**

Lớp:	D23CTPM01
Họ và tên:	Nguyễn Duy Nghĩa
Mã sinh viên:	B23DCCN600
Giảng viên hướng dẫn:	PGS.TS Trần Đình Quế
Học kỳ:	Học kỳ 1 năm học 2026 – 2027

Hà Nội, 2026

Lời mở đầu

Assignment 04 đã cài đặt phép tích chập bằng tay và so sánh ba khung phần mềm. Assignment 05 đặt một câu hỏi khác: từ năm 1998 đến nay các kiến trúc CNN đã thêm những gì, và mỗi thứ thêm vào thực sự đổi được bao nhiêu? Để trả lời, bài nhìn mạng nơ-ron tích chập như một hợp của các hàm, $f = f_L \circ \dots \circ f_1$. Với cách nhìn này, mỗi kiến trúc mới chỉ là một cách thay một hàm, hoặc thay cách nối các hàm với nhau. Ví dụ, ResNet không đổi hàm tích chập; nó thêm x vào đầu ra của hai tầng tích chập.

Báo cáo đi theo đúng năm mục của đề. Chương 1 viết từng khái niệm (neuron, hàm kích hoạt, tích chập, pooling, softmax, lan truyền ngược) thành một hàm NumPy và đối chiếu với PyTorch. Chương 2 đi qua chín kiến trúc từ LeNet đến ViT, mỗi kiến trúc một lớp PyTorch ngắn. Chương 3 mô tả ba tập dữ liệu: CIFAR-10, CIFAR-100 và Diabetes Prediction. Chương 4 xây một CNN cơ bản và ba mô hình phát triển. Mỗi mô hình chỉ thêm đúng một cơ chế so với mô hình trước, rồi cả bốn được huấn luyện trên cả ba tập. Chương 5 so sánh chúng theo chất lượng, chi phí, tốc độ hội tụ, lỗi và biểu diễn.

Kết quả không đi theo câu chuyện “kiến trúc sau luôn tốt hơn kiến trúc trước”. Bước từ CNN cơ bản sang VGG tăng 7,03 điểm accuracy trên CIFAR-10 và 13,23 điểm trên CIFAR-100. Thêm đường tắt của ResNet không đổi gì trên CIFAR-10 và chỉ thêm 1 điểm trên CIFAR-100. Thêm khối chú ý SE không cho lợi ích nhất quán: accuracy đổi $-0,64$ điểm trên CIFAR-10 và $0,15$ điểm trên CIFAR-100. Trên dữ liệu bệnh tiểu đường, cả bốn mô hình có ROC-AUC gần như bằng nhau, và mô hình rẻ nhất là lựa chọn hợp lý. Chương 5 giải thích vì sao: mỗi cơ chế chỉ có ích khi mô hình gặp đúng vấn đề mà cơ chế đó được tạo ra để giải quyết.

Mọi con số trong báo cáo được sinh tự động từ tệp kết quả của notebook, mọi đoạn mã được trích thẳng từ mã nguồn, và mọi hình được vẽ bằng TikZ từ dữ liệu, kể cả ảnh mẫu (vẽ lại từng điểm ảnh). Nếu chạy lại notebook, báo cáo sẽ đổi theo. Phụ lục A ghi các lệnh để tái lập.

Em xin cảm ơn thầy **PGS.TS Trần Đình Quế** đã hướng dẫn học phần và ra đề bài này.

Hà Nội, tháng 9 năm 2026

Nguyễn Duy Nghĩa

Đối chiếu với đề bài

Yêu cầu của đề	Chương	Notebook	Mã nguồn
1. Khái niệm cơ bản của CNN qua hàm và hợp hàm, kèm code	1	00_ham_va_hop_ham	a05/concepts.py
2. Các mô hình phát triển của CNN, hàm và code	2	01_kien_truc_phat_trien	a05/blocks.py
3. Hai tập ảnh và một tập tiểu đường, có link và mô tả	3	02_du_lieu	a05/data.py
4. Code một CNN cơ bản và ba mô hình phát triển	4	03_cifar10, 04_cifar100, 05_diabetes	a05/models.py, a05/train.py
5. So sánh và đánh giá	5	06_so_sanh	a05/metrics.py

Mục lục

Lời mở đầu	i
Mục lục	i
Danh mục Hình vẽ	iv
Danh mục Bảng biểu	v
Danh mục Mã nguồn	vi
1 Khái niệm cơ bản của CNN qua hàm và hợp hàm	1
1.1 Neuron là một hàm affine	1
1.2 Hàm kích hoạt	2
1.3 Tích chập là một hàm	2
1.3.1 Định nghĩa	3
1.3.2 Padding và stride	4
1.3.3 Chia sẻ trọng số: tích chập rẻ hơn tầng nối đầy đủ	4
1.4 Pooling	4
1.5 Đầu ra: softmax và cross-entropy	5
1.6 Một CNN hoàn chỉnh là một hợp hàm	6
1.7 Học là đạo hàm của hàm hợp	7
1.7.1 Quy tắc chuỗi	7
1.7.2 Gradient của tầng tích chập	7
1.7.3 Kiểm chứng bằng sai phân hữu hạn	7
2 Các mô hình phát triển của CNN	9
2.1 LeNet-5 (1998): khuôn mẫu Conv, Pool, FC	9
2.2 AlexNet (2012): sâu hơn, ReLU và Dropout	10
2.3 VGG (2014): chồng các tích chập 3×3	10
2.4 Inception (2014): nhiều cỡ bộ lọc song song	11
2.5 ResNet (2015): đường tắt	11
2.6 DenseNet (2017): nối mọi tầng với nhau	12
2.7 MobileNet (2017): tách tích chập thành hai bước	12
2.8 SE-Net (2017) và CBAM (2018): chú ý	13
2.9 Vision Transformer (2020): bỏ tích chập	14
2.10 Tổng hợp	15
3 Dữ liệu	17
3.1 CIFAR-10	17
3.2 CIFAR-100	17
3.3 Mô tả và tiền xử lý hai tập ảnh	18
3.4 Diabetes Prediction Dataset	20
3.4.1 Mô tả	20
3.4.2 Quan hệ giữa đặc trưng và nhãn	21
3.4.3 Tiền xử lý và dạng tensor	22

4	Một CNN cơ bản và ba mô hình phát triển	24
4.1	Nguyên tắc thiết kế: mỗi bước thêm đúng một cơ chế	24
4.2	Mã nguồn bốn mô hình	25
4.3	Dò shape tầng theo tầng	26
4.4	Bản 1D cho Diabetes	27
4.5	Huấn luyện	28
4.5.1	Cấu hình	28
4.5.2	Vòng lặp huấn luyện	28
4.5.3	Sự cố phân kỳ của M0 và vai trò của BatchNorm	31
4.5.4	Một lượt thí nghiệm	31
5	So sánh và đánh giá	33
5.1	Tiêu chí đánh giá	33
5.2	CIFAR-10	34
5.3	CIFAR-100	35
5.4	Diabetes	36
5.5	Chất lượng và chi phí	38
5.6	Phân tích lỗi	39
5.7	Biểu diễn: bộ lọc lớp đầu và feature map	40
5.8	Tổng hợp: mỗi cơ chế mang lại bao nhiêu	41
6	Kết luận	43
6.1	Trả lời năm mục của đề	43
6.2	Điều rút ra	43
6.3	Hạn chế	43
	Tài liệu tham khảo	44
A	Tái lập kết quả	45
A.1	Môi trường	45
A.2	Các bước	45
A.3	Từ mã nguồn đến báo cáo	45

Danh mục Hình vẽ

1.1	Ba hàm kích hoạt (nét liền) và đạo hàm của chúng (nét đứt)	2
1.2	Tích chập tính tay: mỗi ô đầu ra là tổng tích của bộ lọc với một cửa sổ của ảnh	3
1.3	Ba vị trí đầu tiên của cửa sổ 3×3 khi đệm $P = 1$, bước $S = 2$	4
1.4	Max pooling 2×2 bước 2: mỗi vùng màu cho đúng một ô ở đầu ra	5
1.5	CNN nhỏ viết thành hợp hàm; bốn hàm đầu trích đặc trưng, gradient đi ngược chuỗi	6
2.1	Dòng thời gian các kiến trúc CNN và cơ chế mà mỗi kiến trúc đưa vào	9
2.2	Sáu cơ chế kiến trúc viết thành sơ đồ khối	15
3.1	Mỗi lớp CIFAR-10 một ảnh, vẽ từ 32×32 giá trị RGB	17
3.2	Mỗi siêu lớp CIFAR-100 một ảnh, kèm tên siêu lớp	18
3.3	Phân bố cường độ điểm ảnh theo kênh trên nhánh train	19
3.4	Phân bố HbA1c, đường huyết và tuổi theo hai lớp (mật độ, sau khi bỏ trùng)	21
3.5	Hệ số tương quan Pearson giữa các đặc trưng số, nhị phân và nhãn	22
4.1	Bốn mô hình cho ảnh $32 \times 32 \times 3$; M1, M2, M3 dùng chung khung và chỉ khác nhau ở nội dung của khối	24
4.2	Ảnh test lớp horse và 8 feature map đầu tiên sau tầng tích chập đầu của M0	27
4.3	M0 trên CIFAR-10: không cắt gradient thì phân kỳ khi tốc độ học lên đỉnh	31
5.1	Đường học trên CIFAR-10	35
5.2	Đường học trên CIFAR-100	36
5.3	Đường ROC và Precision–Recall của bốn mô hình trên tập test Diabetes. Đường chấm trên hình PR là tỉ lệ dương, mốc của một bộ phân loại ngẫu nhiên	37
5.4	Đường học trên Diabetes; accuracy tính tại ngưỡng 0,5 của mô hình có pos_weight	37
5.5	Chất lượng (hàng trên) và thời gian mỗi epoch (hàng dưới) của M0–M3 trên ba tập	38
5.6	Accuracy trên tập test theo số tham số (trái) và theo MACs mỗi ảnh, trục log (phải)	38
5.7	Ma trận nhầm lẫn CIFAR-10 của mô hình tốt nhất, % theo hàng	39
5.8	Ma trận nhầm lẫn CIFAR-100 gộp theo 20 siêu lớp, % theo hàng	40
5.9	Bộ lọc lớp đầu và feature map của cùng một ảnh test qua bốn mô hình CIFAR-10	41

Danh mục Bảng biểu

1.1	Shape sau từng hàm của CNN nhỏ (notebook 00)	6
2.1	Shape và số tham số của các khối minh hoạ (notebook 01, tensor giả, lô 2)	15
3.1	Ba tập dữ liệu của bài	17
3.2	Thống kê mô tả bốn đặc trưng số (sau khi bỏ trùng)	21
4.1	Dò shape của M0 (2D, $K = 10$)	27
4.2	Dò shape của M3 (2D, $K = 10$)	27
4.3	Dò shape của M0 bản 1D cho Diabetes	28
4.4	Cấu hình huấn luyện	28
5.1	Kết quả trên tập test CIFAR-10 (10.000 ảnh). In đậm: accuracy cao nhất	34
5.2	Kết quả trên tập test CIFAR-100 (10.000 ảnh, 100 lớp)	35
5.3	Kết quả trên tập test Diabetes (14.422 bệnh nhân). In đậm: giá trị cao nhất mỗi cột	36
5.4	Chênh lệch chất lượng giữa hai mô hình liền kề	41
5.5	Tổng hợp 12 lượt chạy trên tập test	42
A.1	Môi trường chạy thực nghiệm (ghi tự động bởi notebook 06)	45

Danh mục Mã nguồn

1	Neuron tuyến tính	1
2	Hợp hàm theo thứ tự áp dụng	1
3	Hai tầng affine gộp thành một	2
4	ReLU	2
5	Đạo hàm sigmoid viết qua chính sigmoid	2
6	Tích chập 2D bằng NumPy	3
7	Công thức kích thước đầu ra	4
8	Số tham số của tầng tích chập	4
9	Max pooling	5
10	Softmax ổn định số	5
11	Cross-entropy dùng log-sum-exp	5
12	CNN nhỏ viết thẳng thành hợp hàm	6
13	Lan truyền ngược qua tầng tích chập	7
14	Kiểm tra gradient bằng sai phân trung tâm	8
15	LeNet-5	10
16	AlexNet thu nhỏ cho ảnh 32×32	10
17	Khối VGG	11
18	Khối Inception	11
19	Khối Residual	12
20	Khối Dense	12
21	Tích chập depthwise separable	13
22	Khối Squeeze-and-Excitation	13
23	CBAM: chú ý theo kênh rồi theo không gian	14
24	Nhúng ô ảnh của ViT là một tầng tích chập	14
25	Một khối encoder của ViT	14
26	Tách validation phân tầng	18
27	Đọc CIFAR-10/100 từ tệp gốc	19
28	Tăng cường dữ liệu chạy trên GPU	20
29	Sinh lô từ tensor nằm sẵn trên GPU	20
30	Chuẩn bị dữ liệu Diabetes, không rò rỉ từ test	23
31	M0: CNN cơ bản	25
32	Khối hai conv 3×3 , bật tắt đường tắt và SE bằng cờ	25
33	Khung chung của M1, M2, M3	26
34	Dựng mô hình theo tên, cho cả 2D và 1D	26
35	Một epoch huấn luyện hoặc đánh giá	29
36	Huấn luyện nhiều epoch, giữ checkpoint theo val loss	30
37	Huấn luyện một mô hình 2D trên CIFAR	30
38	Huấn luyện một mô hình 1D trên Diabetes	31
39	Một lượt thí nghiệm đầy đủ trên một tập ảnh	32
40	Top- k accuracy	33
41	Ba chỉ số chất lượng cho ảnh	33
42	Đếm MACs bằng forward hook	34
43	Đo thời gian suy luận có đồng bộ GPU	34

Chương 1

Khái niệm cơ bản của CNN qua hàm và hợp hàm

Chương này trả lời mục 1 của đề. Cách nhìn xuyên suốt là: *một mạng nơ-ron tích chập chỉ là một hàm có tham số*, và hàm đó được ghép từ nhiều hàm nhỏ,

$$f_\theta = f_L \circ f_{L-1} \circ \dots \circ f_2 \circ f_1, \quad f_\theta : \mathbb{R}^{C \times H \times W} \rightarrow \mathbb{R}^K.$$

Mỗi khái niệm quen thuộc của CNN (tích chập, hàm kích hoạt, pooling, softmax) là một f_i cụ thể. Huấn luyện là tìm θ làm hàm mất mát nhỏ nhất, và lan truyền ngược chỉ là quy tắc đạo hàm của hàm hợp. Mọi hàm trong chương được viết bằng NumPy thuần trong `a05/concepts.py`, rồi được so với PyTorch trong notebook 00. Kết quả: 14/14 phép kiểm chứng đều khớp.

1.1 Neuron là một hàm affine

Đơn vị nhỏ nhất của mạng nhận vector $\mathbf{x} \in \mathbb{R}^n$ và trả về một số:

$$z = \mathbf{w}^T \mathbf{x} + b = \sum_{i=1}^n w_i x_i + b.$$

Ở đây $\mathbf{w}$ và b là tham số. Gom m neuron thành một tầng thì được hàm affine $\mathbf{x} \mapsto W\mathbf{x} + \mathbf{b}$ với $W \in \mathbb{R}^{m \times n}$. Mã nguồn 1 và 2 viết đúng hai ý này: một neuron, và phép hợp hàm theo thứ tự áp dụng.

```
1 # a05/concepts.py
2 def neuron(x, w, b):
3     """Neuron tuyến tính: z = w · x + b (x, w là vector, b là số)."""
4     return np.dot(w, x) + b
```

Mã nguồn 1. Neuron tuyến tính

```
1 # a05/concepts.py
2 def compose(*fs):
3     """Hợp hàm theo thứ tự áp dụng: compose(f1, f2, f3)(x) = f3(f2(f1(x)))."""
4     return lambda x: reduce(lambda acc, f: f(acc), fs, x)
```

Mã nguồn 2. Hợp hàm theo thứ tự áp dụng

`compose(f1, f2, f3)` trả về hàm $x \mapsto f_3(f_2(f_1(x)))$. Toàn bộ các mạng ở những chương sau đều có dạng này, chỉ khác nhau ở danh sách hàm được truyền vào.

Vì sao cần hàm phi tuyến

Ghép hai tầng affine không cho gì mới:

$$W_2(W_1\mathbf{x} + \mathbf{b}_1) + \mathbf{b}_2 = (W_2W_1)\mathbf{x} + (W_2\mathbf{b}_1 + \mathbf{b}_2).$$

Về phải lại là một tầng affine. Nếu giữa các tầng không có hàm phi tuyến, chong bao nhiêu tầng thì mạng vẫn chỉ biểu diễn được hàm tuyến tính. Notebook 00 kiểm tra điều này bằng số: ghép hai tầng affine ngẫu nhiên cho đúng

kết quả của một tầng gộp, còn khi chèn ReLU vào giữa thì không còn tầng gộp nào khớp.

```
1 # a05/concepts.py
2 def collapse_affine(w1, b1, w2, b2):
3     """Hai tầng affine liên tiếp gộp lại vẫn là một tầng affine:
4     w2 (w1 x + b1) + b2 = (w2 w1) x + (w2 b1 + b2)."""
5     return w2 @ w1, w2 @ b1 + b2
```

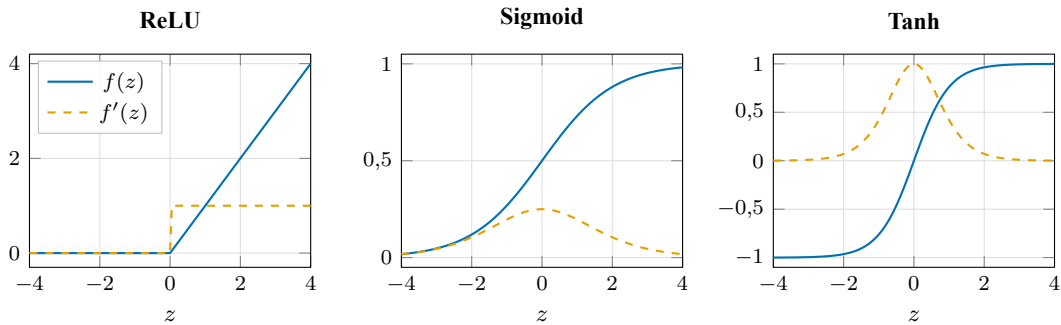
Mã nguồn 3. Hai tầng affine gộp thành một

1.2 Hàm kích hoạt

Hàm kích hoạt σ được áp dụng lên từng phần tử sau mỗi tầng affine hoặc tích chập. Ba hàm hay gặp là

$$\text{ReLU}(z) = \max(0, z), \quad \text{sigmoid}(z) = \frac{1}{1 + e^{-z}}, \quad \tanh(z) = \frac{e^z - e^{-z}}{e^z + e^{-z}}.$$

Khi lan truyền ngược, gradient đi qua mỗi tầng bị nhân với $\sigma'(z)$. Hình 1.1 cho thấy đạo hàm của sigmoid không vượt quá 0,25 và của tanh tiến về 0 ở hai đầu, nên qua nhiều tầng gradient bị thu nhỏ rất nhanh. Đạo hàm của ReLU bằng đúng 1 ở mọi $z > 0$, đó là lý do các CNN từ AlexNet trở đi dùng ReLU.



Hình 1.1. Ba hàm kích hoạt (nét liền) và đạo hàm của chúng (nét đứt)

```
1 # a05/concepts.py
2 def relu(z):
3     return np.maximum(0.0, z)
```

Mã nguồn 4. ReLU

```
1 # a05/concepts.py
2 def sigmoid_grad(z):
3     s = sigmoid(z)
4     return s * (1.0 - s)
```

Mã nguồn 5. Đạo hàm sigmoid viết qua chính sigmoid

1.3 Tích chập là một hàm

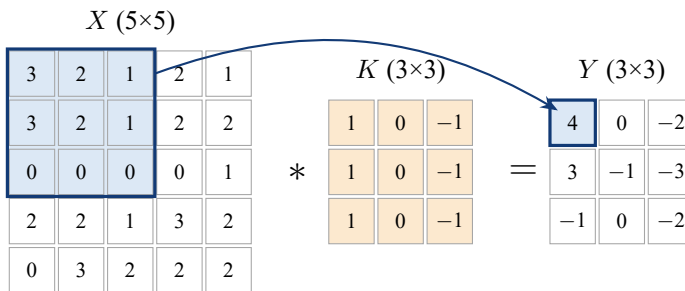
1.3.1 Định nghĩa

Tăng tích chập nhận ảnh $X \in \mathbb{R}^{C \times H \times W}$ và F bộ lọc $K \in \mathbb{R}^{F \times C \times k \times k}$. Kênh ra thứ f tại vị trí (i, j) là

$$Y_{f,i,j} = b_f + \sum_{c=1}^C \sum_{u=0}^{k-1} \sum_{v=0}^{k-1} K_{f,c,u,v} X_{c,iS+u,jS+v},$$

trong đó S là bước trượt (stride). Công thức này thực ra là tương quan chéo (cross-correlation), không lật bộ lọc như định nghĩa tích chập trong toán học. nn.Conv2d của PyTorch cũng tính đúng như vậy. Vì bộ lọc được học từ dữ liệu, việc có lật hay không không ảnh hưởng gì.

Hình 1.2 tính tay một ví dụ của notebook 00: ảnh 5×5 , bộ lọc 3×3 phát hiện biên dọc (cột trái +1, cột phải -1).



$Y_{00} = \sum_{u=0}^2 \sum_{v=0}^2 X_{u,v} K_{u,v}$: tổng tích của vùng tô xanh với bộ lọc. Trượt cửa sổ sang phải một ô được Y_{01} , xuống một ô được Y_{10} .

Hình 1.2. Tích chập tính tay: mỗi ô đầu ra là tổng tích của bộ lọc với một cửa sổ của ảnh

Mã nguồn 6 viết công thức trên thành hai vòng lặp theo vị trí đầu ra. Tại mỗi vị trí, np.tensordot tính tổng tích giữa toàn bộ F bộ lọc và cửa sổ tương ứng. Notebook 00 so hàm này với F.conv2d ở ba cấu hình (stride 1 không đệm, stride 1 đệm 1, stride 2 đệm 1), và cả ba đều khớp.

```
1 # a05/concepts.py
2 def conv2d(X, K, b=None, stride=1, pad=0):
3     """Tích chập 2D (thực chất là cross-correlation, giống nn.Conv2d).
4
5     X: (C, H, W) · K: (F, C, kh, kw) · b: (F,) · trả về (F, Ho, Wo).
6     Mỗi ô đầu ra là tích vô hướng giữa bộ lọc và một cửa sổ của đầu vào.
7     """
8     F, C, kh, kw = K.shape
9     Xp = np.pad(X, ((0, 0), (pad, pad), (pad, pad)))
10    Ho = out_size(X.shape[1], kh, pad, stride)
11    Wo = out_size(X.shape[2], kw, pad, stride)
12    Y = np.zeros((F, Ho, Wo))
13    for i in range(Ho):
14        for j in range(Wo):
15            patch = Xp[:, i * stride:i * stride + kh, j * stride:j * stride + kw]
16            Y[:, i, j] = np.tensordot(K, patch, axes=([1, 2, 3], [0, 1, 2]))
17    if b is not None:
18        Y += b[:, None, None]
19    return Y
```

Mã nguồn 6. Tích chập 2D bằng NumPy

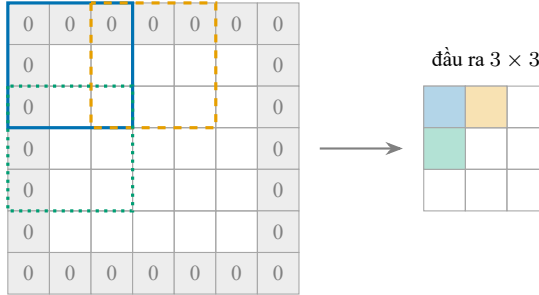
1.3.2 Padding và stride

Đệm (padding) P thêm các ô 0 quanh biên để đầu ra không bị co lại. Bước (stride) S là số ô cửa sổ nhảy mỗi lần. Kích thước đầu ra là

$$H_{ra} = \left\lfloor \frac{H + 2P - K}{S} \right\rfloor + 1.$$

Với $K = 3$, chọn $P = 1, S = 1$ thì giữ nguyên kích thước; đây là cấu hình của mọi tầng tích chập trong Chương 4.

ảnh 5×5 , đệm $P = 1$ (ô xám), bước $S = 2$



$$H_{ra} = \left\lfloor \frac{H + 2P - K}{S} \right\rfloor + 1 = \left\lfloor \frac{5 + 2 \cdot 1 - 3}{2} \right\rfloor + 1 = 3$$

Hình 1.3. Ba vị trí đầu tiên của cửa sổ 3×3 khi đệm $P = 1$, bước $S = 2$

```
1 # a05/concepts.py
2 def out_size(H, K, P, S):
3     """Kích thước đầu ra của tích chập/pooling: floor((H + 2P - K) / S) + 1."""
4     return (H + 2 * P - K) // S + 1
```

Mã nguồn 7. Công thức kích thước đầu ra

1.3.3 Chia sẻ trọng số: tích chập rẻ hơn tầng nối đầy đủ

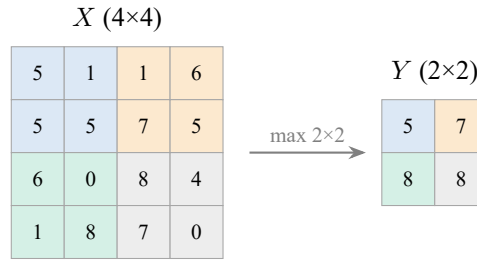
Một bộ lọc được dùng lại ở mọi vị trí của ảnh, nên số tham số của tầng tích chập không phụ thuộc vào kích thước ảnh: nó chỉ là $C_{vào} \cdot C_{ra} \cdot k^2 + C_{ra}$. Với ảnh CIFAR $32 \times 32 \times 3$, nối đầy đủ tới 64 đơn vị cần 196.672 tham số, còn tầng Conv 3×3 ra 64 kênh chỉ cần 1.792, ít hơn khoảng 109,75 lần. Hai con số này được kiểm lại bằng `numel()` của lớp PyTorch tương ứng.

```
1 # a05/concepts.py
2 def count_params_conv(c_in, c_out, k):
3     """Tầng Conv kxk: số tham số không phụ thuộc kích thước ảnh."""
4     return c_in * c_out * k * k + c_out
```

Mã nguồn 8. Số tham số của tầng tích chập

1.4 Pooling

Max pooling thay mỗi cửa sổ bằng giá trị lớn nhất của nó: $Y_{c,i,j} = \max_{0 \leq u,v < s} X_{c,iS+u,jS+v}$. Hàm này không có tham số. Nó giảm kích thước không gian và làm đặc trưng ít nhạy với dịch chuyển nhỏ: dù đặc trưng dịch đi một ô trong cửa sổ, giá trị lớn nhất vẫn giữ nguyên.



Hình 1.4. Max pooling 2×2 bước 2: mỗi vùng màu cho đúng một ô ở đầu ra

```

1 # a05/concepts.py
2 def maxpool2d(X, size=2, stride=2):
3     """Max pooling trên từng kênh:  $X (C, H, W) \rightarrow (C, H_o, W_o)$ ."""
4     C, H, W = X.shape
5     H_o, W_o = out_size(H, size, 0, stride), out_size(W, size, 0, stride)
6     Y = np.empty((C, H_o, W_o))
7     for i in range(H_o):
8         for j in range(W_o):
9             win = X[:, i * stride:i * stride + size, j * stride:j * stride + size]
10            Y[:, i, j] = win.max(axis=(1, 2))
11     return Y

```

Mã nguồn 9. Max pooling

1.5 Đầu ra: softmax và cross-entropy

Sau các tầng trích đặc trưng, tensor được làm phẳng (flatten) rồi qua một tầng nối đầy đủ để cho ra K logits $\mathbf{z} \in \mathbb{R}^K$. Softmax biến logits thành phân phối xác suất, và hàm mất mát cross-entropy phạt xác suất thấp gán cho nhãn đúng y :

$$\hat{y}_k = \frac{e^{z_k}}{\sum_j e^{z_j}}, \quad L(\mathbf{z}, y) = -\log \hat{y}_y = \log \sum_j e^{z_j} - z_y.$$

Viết thẳng công thức này thì e^{1000} sẽ tràn số. Trừ đi $m = \max_j z_j$ ở cả tử và mẫu không làm đổi kết quả, nhưng đưa mọi số mũ về ≤ 0 . Notebook 00 tính được $\text{softmax}(1000, 1001, 1002)$ mà không tràn, và giá trị cross-entropy khớp `F.cross_entropy`.

```

1 # a05/concepts.py
2 def softmax(z):
3     """Softmax ổn định số: trừ max trước khi lấy mũ, kết quả không đổi."""
4     e = np.exp(z - np.max(z))
5     return e / e.sum()

```

Mã nguồn 10. Softmax ổn định số

```

1 # a05/concepts.py
2 def cross_entropy(z, y):
3     """Cross-entropy từ logits  $z$  với nhãn nguyên  $y$ , dùng log-sum-exp ổn định."""
4     m = np.max(z)
5     return m + np.log(np.exp(z - m).sum()) - z[y]

```

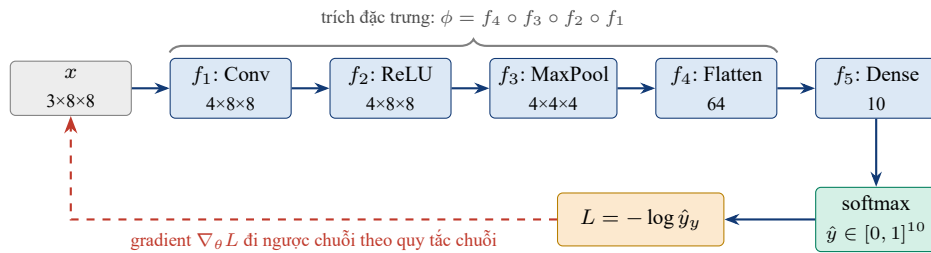
Mã nguồn 11. Cross-entropy dùng log-sum-exp

1.6 Một CNN hoàn chỉnh là một hợp hàm

Ghép các hàm ở trên lại, ta được một CNN nhỏ:

$$f = \text{dense} \circ \text{flatten} \circ \text{maxpool} \circ \text{ReLU} \circ \text{conv}.$$

Hình 1.5 vẽ hợp hàm này cùng shape tensor sau từng hàm. Bảng 1.1 là bản in thật của notebook 00 khi chạy trên ảnh $3 \times 8 \times 8$ với bốn bộ lọc.



Hình 1.5. CNN nhỏ viết thành hợp hàm; bốn hàm đầu trích đặc trưng, gradient đi ngược chuỗi

Bảng 1.1. Shape sau từng hàm của CNN nhỏ (notebook 00)

Hàm	Tên	Shape
–	input	$3 \times 8 \times 8$
f_1	conv	$4 \times 8 \times 8$
f_2	relu	$4 \times 8 \times 8$
f_3	maxpool	$4 \times 4 \times 4$
f_4	flatten	64
f_5	dense	10

```

1 # a05/concepts.py
2 def tiny_cnn_forward(X, p):
3     """CNN nhỏ viết thẳng thành hợp hàm, ghi lại shape sau từng hàm.
4
5     f = softmax o dense o flatten o maxpool o relu o conv
6     p: dict có 'K', 'b' (conv) và 'W', 'c' (dense). Trả về (Logits, trace).
7     """
8     layers = [
9         ("conv", lambda x: conv2d(x, p["K"], p["b"], stride=1, pad=1)),
10        ("relu", relu),
11        ("maxpool", maxpool2d),
12        ("flatten", flatten),
13        ("dense", lambda x: p["W"] @ x + p["c"]),
14    ]
15    trace = [("input", X.shape)]
16    for name, f in layers:
17        X = f(X)
18        trace.append((name, X.shape))
19    return X, trace

```

Mã nguồn 12. CNN nhỏ viết thẳng thành hợp hàm

Hàm này nhận đúng một danh sách (tên, hàm) và áp dụng lần lượt, hay chính là compose có kèm theo việc ghi

shape. Với cùng trọng số, logits của nó khớp với `nn.Sequential(Conv2d, ReLU, MaxPool2d, Flatten, Linear)`. Như vậy `nn.Sequential` cũng không làm gì hơn một phép hợp hàm.

1.7 Học là đạo hàm của hàm hợp

1.7.1 Quy tắc chuỗi

Huấn luyện tìm θ làm nhỏ rủi ro thực nghiệm $\mathcal{L}(\theta) = \frac{1}{N} \sum_n L(f_\theta(\mathbf{x}_n), y_n)$ bằng gradient descent $\theta \leftarrow \theta - \eta \nabla_\theta \mathcal{L}$. Vì f_θ là hàm hợp, gradient theo tham số của tầng l được tính bằng quy tắc chuỗi, đi ngược từ đầu ra:

$$\frac{\partial L}{\partial \theta_l} = \frac{\partial L}{\partial \mathbf{a}_L} \frac{\partial \mathbf{a}_L}{\partial \mathbf{a}_{L-1}} \cdots \frac{\partial \mathbf{a}_{l+1}}{\partial \mathbf{a}_l} \frac{\partial \mathbf{a}_l}{\partial \theta_l}, \quad \mathbf{a}_l = f_l(\mathbf{a}_{l-1}).$$

Mỗi tầng chỉ cần biết hai việc: nhận gradient $G = \partial L / \partial Y$ từ tầng sau, và trả về $\partial L / \partial X$ cho tầng trước, kèm gradient theo tham số của chính nó.

1.7.2 Gradient của tầng tích chập

Vì $Y_{f,i,j}$ tuyến tính theo cả K lẫn X , đạo hàm của nó chỉ là cộng dồn:

$$\frac{\partial L}{\partial K_{f,c,u,v}} = \sum_{i,j} G_{f,i,j} X_{c,iS+u,jS+v}, \quad \frac{\partial L}{\partial X_{c,p,q}} = \sum_f \sum_{\substack{i,j,u,v \\ iS+u=p, jS+v=q}} G_{f,i,j} K_{f,c,u,v}, \quad \frac{\partial L}{\partial b_f} = \sum_{i,j} G_{f,i,j}.$$

Mã nguồn 13 duyệt lại đúng các cửa sổ mà lượt thuận đã dùng và cộng dồn theo hai công thức đầu.

```

1 # a05/concepts.py
2 def conv2d_backward(X, K, dY, stride=1, pad=0):
3     """Lan truyền ngược của conv2d theo quy tắc chuỗi.
4
5     Mỗi ô đầu ra  $Y[f,i,j]$  phụ thuộc tuyến tính vào một cửa sổ của  $X$  và vào  $K[f]$ ,
6     nên gradient chỉ là cộng dồn  $dY[f,i,j]$  nhân với phần tử tương ứng.
7     Trả về  $(dX, dK, dB)$ .
8     """
9     F, C, kh, kw = K.shape
10    Xp = np.pad(X, ((0, 0), (pad, pad), (pad, pad)))
11    dXp = np.zeros_like(Xp)
12    dK = np.zeros_like(K)
13    _, Ho, Wo = dY.shape
14    for i in range(Ho):
15        for j in range(Wo):
16            rs, cs = i * stride, j * stride
17            patch = Xp[:, rs:rs + kh, cs:cs + kw]
18            dK += dY[:, i, j][:, None, None, None] * patch[None]
19            dXp[:, rs:rs + kh, cs:cs + kw] += np.tensordot(dY[:, i, j], K, axes=(0, 0))
20    dX = dXp[:, pad:pad + X.shape[1], pad:pad + X.shape[2]]
21    return dX, dK, dY.sum(axis=(1, 2))

```

Mã nguồn 13. Lan truyền ngược qua tầng tích chập

1.7.3 Kiểm chứng bằng sai phân hữu hạn

Một gradient viết tay chỉ đáng tin khi đã được đối chiếu với định nghĩa đạo hàm. Hàm `grad_check` nhích từng phần tử $\pm \varepsilon$ và tính sai phân trung tâm $(L(\theta + \varepsilon) - L(\theta - \varepsilon)) / 2\varepsilon$. Trên ảnh $3 \times 9 \times 9$, bốn bộ lọc, stride 2, đệm 1, sai số tương đối là $5,34 \times 10^{-10}$ với $\partial L / \partial K$, $6,64 \times 10^{-10}$ với $\partial L / \partial X$ và $7,16 \times 10^{-10}$ với $\partial L / \partial b$, đều dưới ngưỡng 10^{-6} . Một bước cập nhật $K \leftarrow K - \eta \partial L / \partial K$ với $\eta = 0,001$ làm L giảm từ 45,95 xuống 43,73, đúng hướng mà gradient chỉ ra.

```

1 # a05/concepts.py
2 def grad_check(f, x, grad, eps=1e-6):
3     """So gradient giải tích với sai phân trung tâm (f(x+e) - f(x-e)) / 2e.
4
5     Trả về sai số tương đối lớn nhất ||g_số - g_giải_tích|| / ||g_số + g_giải_tích||.
6     """
7     num = np.zeros_like(x)
8     it = np.nditer(x, flags=["multi_index"])
9     for _ in it:
10         idx = it.multi_index
11         old = x[idx]
12         x[idx] = old + eps
13         fp = f(x)
14         x[idx] = old - eps
15         fm = f(x)
16         x[idx] = old
17         num[idx] = (fp - fm) / (2 * eps)
18     return np.linalg.norm(num - grad) / max(np.linalg.norm(num + grad), 1e-12)

```

Mã nguồn 14. Kiểm tra gradient bằng sai phân trung tâm

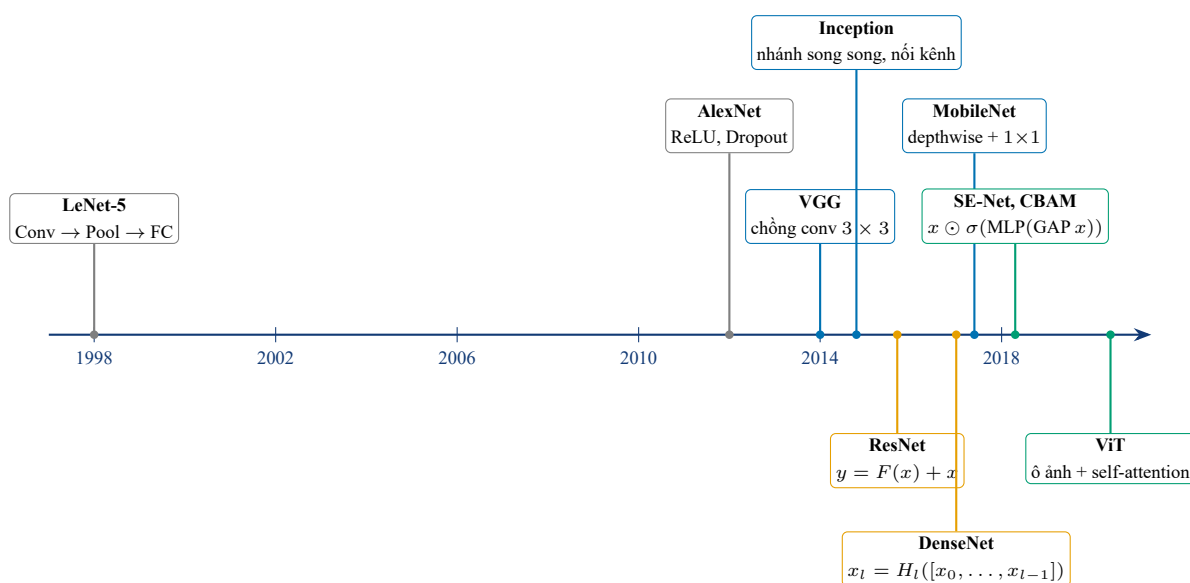
Tóm tắt Chương 1

Mỗi thành phần của CNN là một hàm: tích chập là hàm tuyến tính có chia sẻ trọng số, ReLU là hàm phi tuyến đặt giữa các tầng tuyến tính để chúng không gộp lại thành một tầng, pooling là hàm lấy cực đại cục bộ, softmax và cross-entropy biến logits thành mất mát. CNN là hợp của các hàm đó, và việc học là tính đạo hàm của hàm hợp bằng quy tắc chuỗi. Toàn bộ 14 hàm viết tay khớp với PyTorch.

Chương 2

Các mô hình phát triển của CNN

Chương 1 cho thấy CNN là hợp của các hàm. Từ năm 1998 đến nay, mỗi kiến trúc mới thường chỉ đổi *một* trong các hàm ấy, hoặc đổi cách chúng được nối với nhau, để sửa một điểm yếu cụ thể của kiến trúc trước. Chương này đi qua chín mốc chính (Hình 2.1). Với mỗi mốc, chương nêu vấn đề nó giải quyết, viết cơ chế mới thành công thức, rồi đưa ra một lớp PyTorch ngắn trong `a05/blocks.py`. Notebook 01 chạy mỗi lớp một lượt trên tensor giả để kiểm tra shape và đếm tham số; mục này không huấn luyện.



Hình 2.1. Dòng thời gian các kiến trúc CNN và cơ chế mà mỗi kiến trúc đưa vào

2.1 LeNet-5 (1998): khuôn mẫu Conv, Pool, FC

LeCun và cộng sự [1] đặt ra khuôn mẫu mà mọi CNN sau này vẫn giữ: vài lần lặp (tích chập, hàm kích hoạt, pooling) để trích đặc trưng, rồi các tầng nối đầy đủ để phân loại,

$$f = \text{FC}_3 \circ \tanh \circ \text{FC}_2 \circ \tanh \circ \text{FC}_1 \circ \text{flatten} \circ (\text{pool} \circ \tanh \circ \text{conv}_{5 \times 5})^2.$$

Bản dựng lại cho ảnh $32 \times 32 \times 3$ có 62.006 tham số.

```

1 # a05/blocks.py
2 class LeNet5(nn.Module):
3     """1998 - Conv -> Pool -> Conv -> Pool -> FC -> FC -> FC."""
4
5     def __init__(self, in_ch=3, n_classes=10):
6         super().__init__()
7         self.features = nn.Sequential(
8             nn.Conv2d(in_ch, 6, 5), nn.Tanh(), nn.AvgPool2d(2),
9             nn.Conv2d(6, 16, 5), nn.Tanh(), nn.AvgPool2d(2))
10        self.head = nn.Sequential(
11            nn.Flatten(), nn.Linear(16 * 5 * 5, 120), nn.Tanh(),
12            nn.Linear(120, 84), nn.Tanh(), nn.Linear(84, n_classes))
13
14    def forward(self, x):
15        return self.head(self.features(x))

```

Mã nguồn 15. LeNet-5

2.2 AlexNet (2012): sâu hơn, ReLU và Dropout

LeNet dùng tanh, có đạo hàm nhỏ ở hai đầu. Khi mạng sâu thêm, tích của nhiều đạo hàm nhỏ làm gradient ở các tầng đầu gần như bằng 0. AlexNet [2] thay tanh bằng $\text{ReLU}(z) = \max(0, z)$, có đạo hàm bằng 1 trên miền dương. Để chống quá khớp ở các tầng nối đầy đủ lớn, AlexNet dùng Dropout: khi huấn luyện, mỗi đơn vị bị đặt về 0 với xác suất p , tức nhân đầu ra với mặt nạ ngẫu nhiên $\mathbf{m} \sim \text{Bernoulli}(1 - p)$. Bản thu nhỏ trong `blocks.py` có 1.575.626 tham số, phần lớn nằm ở tầng nối đầy đủ đầu tiên.

```

1 # a05/blocks.py
2 class AlexNetMini(nn.Module):
3     """2012 - sâu hơn, ReLU thay tanh, Dropout ở phần nối đầy đủ."""
4
5     def __init__(self, in_ch=3, n_classes=10):
6         super().__init__()
7         self.features = nn.Sequential(
8             nn.Conv2d(in_ch, 64, 5, padding=2), nn.ReLU(), nn.MaxPool2d(2),
9             nn.Conv2d(64, 128, 3, padding=1), nn.ReLU(), nn.MaxPool2d(2),
10            nn.Conv2d(128, 192, 3, padding=1), nn.ReLU(),
11            nn.Conv2d(192, 128, 3, padding=1), nn.ReLU(), nn.MaxPool2d(2))
12        self.head = nn.Sequential(
13            nn.Flatten(), nn.Dropout(0.5), nn.Linear(128 * 4 * 4, 512), nn.ReLU(),
14            nn.Dropout(0.5), nn.Linear(512, n_classes))
15
16    def forward(self, x):
17        return self.head(self.features(x))

```

Mã nguồn 16. AlexNet thu nhỏ cho ảnh 32×32

2.3 VGG (2014): chồng các tích chập 3×3

Simonyan và Zisserman [3] chỉ dùng một loại bộ lọc: 3×3 . Hai tầng 3×3 liên tiếp có vùng nhìn 5×5 , giống một tầng 5×5 , nhưng với C kênh vào và ra thì cần $2 \cdot 9C^2 = 18C^2$ trọng số thay vì $25C^2$, đồng thời có thêm một hàm phi tuyến ở giữa. Với $C = 64$, notebook 01 đếm được 102.400 trọng số cho một conv 5×5 và 73.728 cho hai conv 3×3 , ít hơn 28,00 %. Một khối VGG là hàm pool $\circ (\text{ReLU} \circ \text{conv}_{3 \times 3})^n$.

```

1 # a05/blocks.py
2 class VGGBlock(nn.Module):
3     """2014 - chồng n conv 3x3: hai conv 3x3 nhìn được vùng 5x5 với ít tham số hơn."""
4
5     def __init__(self, c_in, c_out, n_convs=2):
6         super().__init__()
7         layers = []
8         for i in range(n_convs):
9             layers += [nn.Conv2d(c_in if i == 0 else c_out, c_out, 3, padding=1), nn.ReLU()]
10        self.body = nn.Sequential(*layers, nn.MaxPool2d(2))
11
12    def forward(self, x):
13        return self.body(x)

```

Mã nguồn 17. Khối VGG

2.4 Inception (2014): nhiều cỡ bộ lọc song song

Chọn bộ lọc 3×3 hay 5×5 là chọn trước một cỡ vùng nhìn. Inception [4] không chọn: nó chạy song song bốn nhánh rồi nối kết quả theo chiều kênh,

$$y = \text{concat}(f_{1 \times 1}(x), f_{3 \times 3}(x), f_{5 \times 5}(x), f_{\text{pool}}(x)).$$

Để nhánh 3×3 và 5×5 không quá đắt, một conv 1×1 giảm số kênh trước đó. Conv 1×1 là một tầng nối đầy đủ áp dụng riêng tại từng điểm ảnh, trộn các kênh mà không nhìn sang điểm bên cạnh.

```

1 # a05/blocks.py
2 class InceptionBlock(nn.Module):
3     """2014 - bốn nhánh song song, y = concat(f1(x), f3(x), f5(x), fpool(x))."""
4
5     def __init__(self, c_in, c1, c3, c5, cp):
6         super().__init__()
7         self.b1 = nn.Conv2d(c_in, c1, 1)
8         self.b3 = nn.Sequential(nn.Conv2d(c_in, c3 // 2, 1), nn.ReLU(), nn.Conv2d(c3 // 2, c3, 3, padding=1))
9         self.b5 = nn.Sequential(nn.Conv2d(c_in, c5 // 2, 1), nn.ReLU(), nn.Conv2d(c5 // 2, c5, 5, padding=2))
10        self.bp = nn.Sequential(nn.MaxPool2d(3, stride=1, padding=1), nn.Conv2d(c_in, cp, 1))
11
12    def forward(self, x):
13        return torch.relu(torch.cat([self.b1(x), self.b3(x), self.b5(x), self.bp(x)], dim=1))

```

Mã nguồn 18. Khối Inception

2.5 ResNet (2015): đường tắt

Khi VGG được làm sâu hơn khoảng 20 tầng, sai số trên cả tập *huấn luyện* lại tăng lên. Đây không phải quá khớp mà là mạng khó tối ưu. He và cộng sự [5] đổi mục tiêu của mỗi khối: thay vì học trực tiếp ánh xạ $H(x)$, khối học phần dư $F(x) = H(x) - x$,

$$y = \text{ReLU}(F(x) + x).$$

Hai hệ quả trực tiếp của công thức này. Thứ nhất, nếu $F \equiv 0$ thì khối là ánh xạ đồng nhất, nên thêm một khối không thể làm mạng tệ đi. Notebook 01 kiểm tra điều này bằng cách đặt γ của BatchNorm cuối về 0: đầu ra khớp đầu vào (đúng). Thứ hai, đạo hàm $\partial y / \partial x = \partial F / \partial x + I$ luôn có thành phần I , nên gradient đi thẳng qua đường tắt mà không bị thu nhỏ qua từng tầng. Khối minh hoạ với 64 kênh có 73.984 tham số.

```

1 # a05/blocks.py
2 class ResidualBlock(nn.Module):
3     """2015 - y = ReLU(F(x) + x): mạng chỉ cần học phần dư F(x) = H(x) - x."""
4
5     def __init__(self, c):
6         super().__init__()
7         self.F = nn.Sequential(
8             nn.Conv2d(c, c, 3, padding=1, bias=False), nn.BatchNorm2d(c), nn.ReLU(),
9             nn.Conv2d(c, c, 3, padding=1, bias=False), nn.BatchNorm2d(c))
10
11     def forward(self, x):
12         return torch.relu(self.F(x) + x)

```

Mã nguồn 19. Khối Residual

2.6 DenseNet (2017): nối mọi tầng với nhau

DenseNet [6] đẩy ý tưởng đường tắt đi xa hơn. Mỗi tầng nhận *tất cả* đầu ra trước nó, nối theo kênh chứ không cộng:

$$x_l = H_l([x_0, x_1, \dots, x_{l-1}]).$$

Mỗi tầng chỉ thêm k kênh mới (tốc độ tăng trưởng, *growth rate*), nên các tầng có thể rất hẹp. Khối trong notebook bắt đầu từ 16 kênh, qua 4 tầng với $k = 12$, ra $16 + 4 \cdot 12 = 64$ kênh, dùng 14.960 tham số.

```

1 # a05/blocks.py
2 class DenseBlock(nn.Module):
3     """2017 - x_l = H_l([x_0, x_1, ..., x_{l-1}]): mỗi tầng nhận mọi đầu ra trước đó."""
4
5     def __init__(self, c_in, growth, n_layers):
6         super().__init__()
7         self.layers = nn.ModuleList(
8             nn.Sequential(nn.BatchNorm2d(c_in + i * growth), nn.ReLU(),
9                           nn.Conv2d(c_in + i * growth, growth, 3, padding=1, bias=False))
10             for i in range(n_layers))
11
12     def forward(self, x):
13         feats = [x]
14         for layer in self.layers:
15             feats.append(layer(torch.cat(feats, dim=1)))
16         return torch.cat(feats, dim=1)

```

Mã nguồn 20. Khối Dense

2.7 MobileNet (2017): tách tích chập thành hai bước

Một tầng conv 3×3 thường vừa lọc theo không gian vừa trộn kênh, tốn $9 C_{in} C_{out}$ trọng số. MobileNet [7] tách hai việc đó: *depthwise* lọc từng kênh riêng ($9 C_{in}$ trọng số), rồi *pointwise* 1×1 trộn kênh ($C_{in} C_{out}$). Với $64 \rightarrow 128$ kênh, conv thường cần 73.728 trọng số, còn bản tách chỉ cần 8.768, ít hơn 8,41 lần.

```

1 # a05/blocks.py
2 class DepthwiseSeparable(nn.Module):
3     """2017 – tách conv thường thành depthwise (từng kênh) rồi pointwise 1x1."""
4
5     def __init__(self, c_in, c_out):
6         super().__init__()
7         self.depthwise = nn.Conv2d(c_in, c_in, 3, padding=1, groups=c_in, bias=False)
8         self.pointwise = nn.Conv2d(c_in, c_out, 1, bias=False)
9
10    def forward(self, x):
11        return torch.relu(self.pointwise(torch.relu(self.depthwise(x))))

```

Mã nguồn 21. Tích chập depthwise separable

2.8 SE-Net (2017) và CBAM (2018): chú ý

Các kiến trúc trên đều coi mọi kênh đặc trưng như nhau. Squeeze-and-Excitation [8] cho mạng tự học một trọng số cho từng kênh, tính từ chính dữ liệu đầu vào:

$$\mathbf{s} = \text{GAP}(x) \in \mathbb{R}^C, \quad \mathbf{w} = \sigma(W_2 \text{ReLU}(W_1 \mathbf{s})) \in (0, 1)^C, \quad y_c = w_c x_c.$$

Bước *squeeze* nén mỗi kênh về một số bằng trung bình toàn cục, bước *excitation* là một MLP nhỏ với tỉ lệ nén r ($W_1 \in \mathbb{R}^{C/r \times C}$). Khối này rất rẻ: với 64 kênh chỉ thêm 580 tham số. Trên tensor ngẫu nhiên, notebook 01 đo được các hệ số nằm trong khoảng (0,3795; 0,6248). CBAM [9] làm thêm một bước chú ý theo *không gian*: gộp các kênh bằng trung bình và max, qua một conv 7×7 và sigmoid để có trọng số cho từng vị trí.

```

1 # a05/blocks.py
2 class SEBlock(nn.Module):
3     """2017 –  $y = x \odot \sigma(W_2 \text{ReLU}(W_1 \text{GAP}(x)))$ : mỗi kênh nhận một trọng số trong (0, 1)."""
4
5     def __init__(self, c, r=16):
6         super().__init__()
7         h = max(c // r, 4)
8         self.fc = nn.Sequential(nn.Linear(c, h), nn.ReLU(), nn.Linear(h, c), nn.Sigmoid())
9
10    def forward(self, x): # chạy cho cả (B, C, L) lẫn (B, C, H, W)
11        spatial = tuple(range(2, x.ndim))
12        w = self.fc(x.mean(dim=spatial))
13        return x * w.view(*w.shape, *[1] * len(spatial))

```

Mã nguồn 22. Khối Squeeze-and-Excitation

```

1 # a05/blocks.py
2 class CBAM(nn.Module):
3     """2018 - chú ý theo kênh (như SE, thêm nhánh max) rồi chú ý theo không gian."""
4
5     def __init__(self, c, r=16):
6         super().__init__()
7         h = max(c // r, 4)
8         self.mlp = nn.Sequential(nn.Linear(c, h), nn.ReLU(), nn.Linear(h, c))
9         self.spatial = nn.Conv2d(2, 1, 7, padding=3)
10
11     def forward(self, x):
12         ca = torch.sigmoid(self.mlp(x.mean(dim=(2, 3))) + self.mlp(x.amax(dim=(2, 3))))
13         x = x * ca[:, :, None, None]
14         sa = torch.sigmoid(self.spatial(torch.cat([x.mean(1, keepdim=True), x.amax(1, keepdim=True)], 1)))
15         return x * sa

```

Mã nguồn 23. CBAM: chú ý theo kênh rồi theo không gian

2.9 Vision Transformer (2020): bỏ tích chập

ViT [10] cắt ảnh thành các ô $p \times p$, chiếu mỗi ô thành một vector (*token*), rồi dùng self-attention để mọi ô nhìn thấy mọi ô khác ngay từ tầng đầu. Tích chập chỉ nhìn một vùng lân cận nhỏ; self-attention thì tính trọng số giữa hai ô bất kỳ:

$$\text{Attn}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d}}\right) V.$$

Điều thú vị là bước nhúng ô chính là một tầng conv có kích thước bộ lọc bằng bước trượt bằng p (Mã nguồn 24). Với $p = 4$, ảnh 32×32 thành $8 \cdot 8 = 64$ token.

```

1 # a05/blocks.py
2 class PatchEmbed(nn.Module):
3     """2020 (ViT) - cắt ảnh thành các ô p x p, chiếu mỗi ô thành một vector d chiều."""
4
5     def __init__(self, in_ch=3, patch=4, dim=64):
6         super().__init__()
7         self.proj = nn.Conv2d(in_ch, dim, patch, stride=patch)
8
9     def forward(self, x):
10         return self.proj(x).flatten(2).transpose(1, 2) # (B, số ô, d)

```

Mã nguồn 24. Nhúng ô ảnh của ViT là một tầng tích chập

```

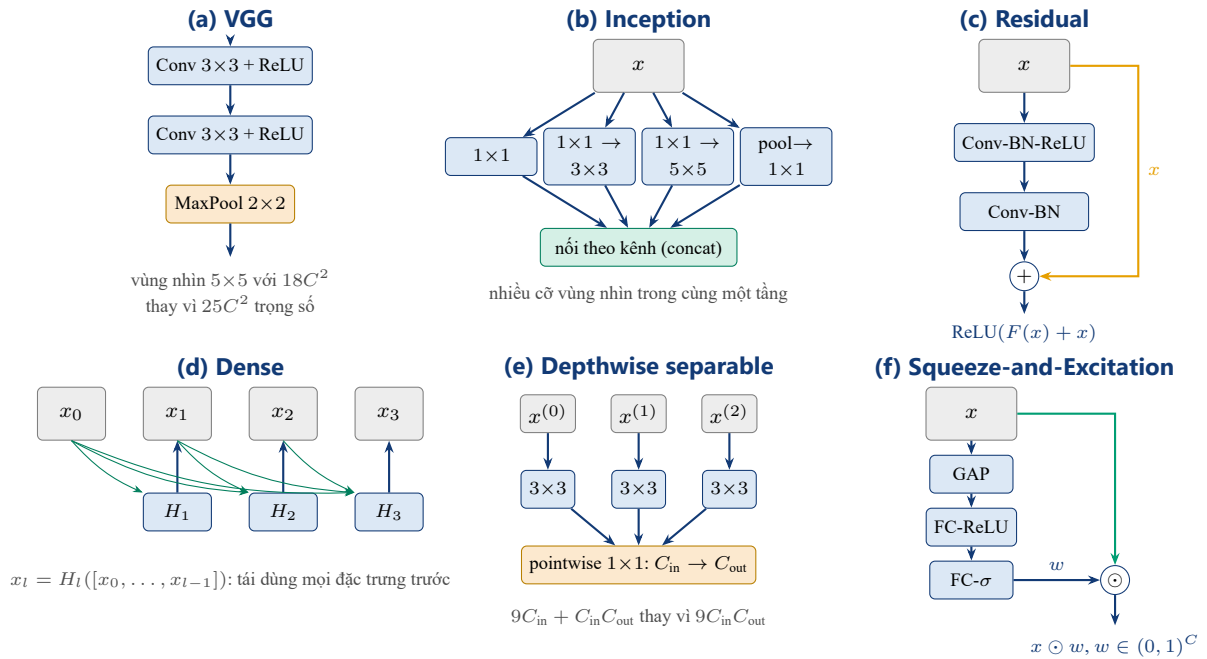
1 # a05/blocks.py
2 class TinyViTBlock(nn.Module):
3     """Một khối encoder: z = z + MHA(LN(z)); z = z + MLP(LN(z))."""
4
5     def __init__(self, dim=64, heads=4):
6         super().__init__()
7         self.ln1, self.ln2 = nn.LayerNorm(dim), nn.LayerNorm(dim)
8         self.attn = nn.MultiheadAttention(dim, heads, batch_first=True)
9         self.mlp = nn.Sequential(nn.Linear(dim, 2 * dim), nn.GELU(), nn.Linear(2 * dim, dim))
10
11     def forward(self, z):
12         h = self.ln1(z)
13         z = z + self.attn(h, h, h, need_weights=False)[0]
14         return z + self.mlp(self.ln2(z))

```

Mã nguồn 25. Một khối encoder của ViT

2.10 Tổng hợp

Hình 2.2 vẽ sáu cơ chế chính dưới dạng sơ đồ khối, và Bảng 2.1 ghi shape cùng số tham số mà notebook 01 đo được cho từng khối.



Hình 2.2. Sáu cơ chế kiến trúc viết thành sơ đồ khối

Bảng 2.1. Shape và số tham số của các khối minh hoạ (notebook 01, tensor giả, lô 2)

Khối minh hoạ	Năm	Vào (C×H×W)	Ra	Tham số
LeNet-5	1998	$3 \times 32 \times 32$	10	62.006
AlexNet (thu nhỏ)	2012	$3 \times 32 \times 32$	10	1.575.626
Khối VGG	2014	$3 \times 32 \times 32$	$64 \times 16 \times 16$	38.720
Khối Inception	2014	$64 \times 16 \times 16$	$128 \times 16 \times 16$	27.432
Khối Residual	2015	$64 \times 16 \times 16$	$64 \times 16 \times 16$	73.984
Khối Dense (4 tầng)	2017	$16 \times 32 \times 32$	$64 \times 32 \times 32$	14.960
Depthwise separable	2017	$64 \times 16 \times 16$	$128 \times 16 \times 16$	8.768
SE	2017	$64 \times 16 \times 16$	$64 \times 16 \times 16$	580
CBAM	2018	$64 \times 16 \times 16$	$64 \times 16 \times 16$	679
ViT: nhúng ô	2020	$3 \times 32 \times 32$	64×64	3.136
ViT: khối encoder	2020	64×64	64×64	33.472

Ba mô hình phát triển được chọn cho Chương 4

Trong chín mốc trên, ba cơ chế được đem đi huấn luyện ở Chương 4: chồng conv 3×3 kèm BatchNorm (VGG), đường tắt (ResNet) và chú ý theo kênh (SE). Lý do chọn ba cơ chế này: mỗi cơ chế thêm được vào

mô hình trước mà không phải đổi phần còn lại, nên mỗi bước so sánh chỉ khác đúng một cơ chế. Inception, DenseNet và ViT đổi cả khung kiến trúc, nên không so sánh theo cách này được.

Chương 3

Dữ liệu

Mục 3 của đề yêu cầu hai tập ảnh và một tập về bệnh tiểu đường, có đường dẫn và mô tả. Bảng 3.1 tóm tắt ba tập được dùng. Hai đường dẫn đã được kiểm tra còn truy cập được (HTTP 200) ngày 25/9/2026.

Bảng 3.1. Ba tập dữ liệu của bài

Tập	Nguồn	Quy mô
CIFAR-10	https://www.cs.toronto.edu/~kriz/cifar.html [11]	45.000 + 5.000 + 10.000 ảnh $32 \times 32 \times 3$, 10 lớp
CIFAR-100	cùng trang trên, tệp <code>cifar-100-python.tar.gz</code>	45.000 + 5.000 + 10.000 ảnh $32 \times 32 \times 3$, 100 lớp mịn, 20 siêu lớp
Diabetes	https://www.kaggle.com/datasets/iammustafatz/diabetes-prediction-dataset [12]	100.000 dòng, 8 đặc trưng, nhãn nhị phân

Về tên “MNIST-100” trong đề

Đề ghi hai tập ảnh là CIFAR-10 và “MNIST-100”. Không có bộ dữ liệu chuẩn nào mang tên MNIST-100. Bài hiểu đây là **CIFAR-100**: cùng tác giả, cùng kích thước ảnh và cùng định dạng tệp với CIFAR-10, chỉ khác số lớp (100 thay vì 10). Cặp CIFAR-10/CIFAR-100 cũng cho một phép thử có ý nghĩa: cùng kiến trúc, cùng cấu hình, chỉ tăng số lớp lên mười lần.

3.1 CIFAR-10

CIFAR-10 [11] gồm 60.000 ảnh màu tự nhiên kích thước 32×32 thuộc 10 lớp. Mỗi lớp có đúng 5.000 ảnh huấn luyện và 1.000 ảnh kiểm tra, tức hoàn toàn cân bằng. Hình 3.1 là ảnh huấn luyện đầu tiên của mỗi lớp, vẽ lại từng điểm ảnh bằng pgfplots từ tệp dữ liệu mà notebook 02 xuất ra.

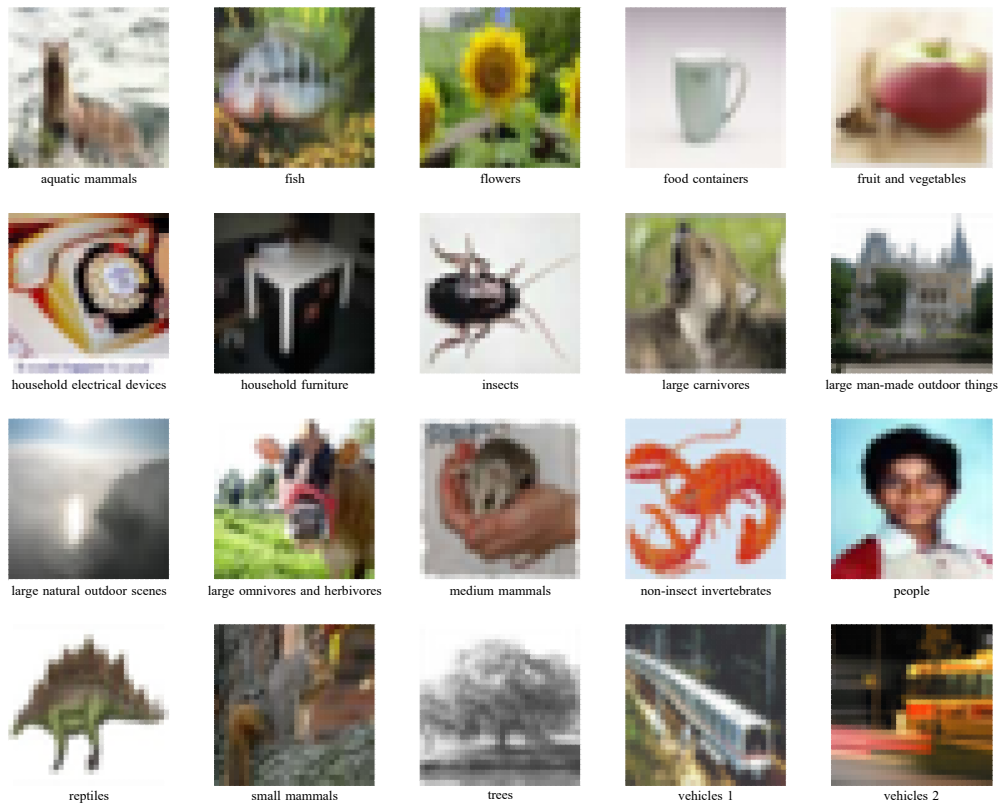


Hình 3.1. Mỗi lớp CIFAR-10 một ảnh, vẽ từ 32×32 giá trị RGB

3.2 CIFAR-100

CIFAR-100 có cùng 60.000 ảnh 32×32 nhưng chia thành 100 lớp mịn, mỗi lớp chỉ 500 ảnh huấn luyện, ít hơn CIFAR-10 mười lần. 100 lớp mịn được gom thành 20 siêu lớp, mỗi siêu lớp đúng 5 lớp mịn. Mô hình được huấn

luyện trên nhãn mịn; siêu lớp chỉ dùng ở Chương 5 để vẽ ma trận nhầm lẫn ở mức 20 nhóm cho dễ đọc.



Hình 3.2. Mỗi siêu lớp CIFAR-100 một ảnh, kèm tên siêu lớp

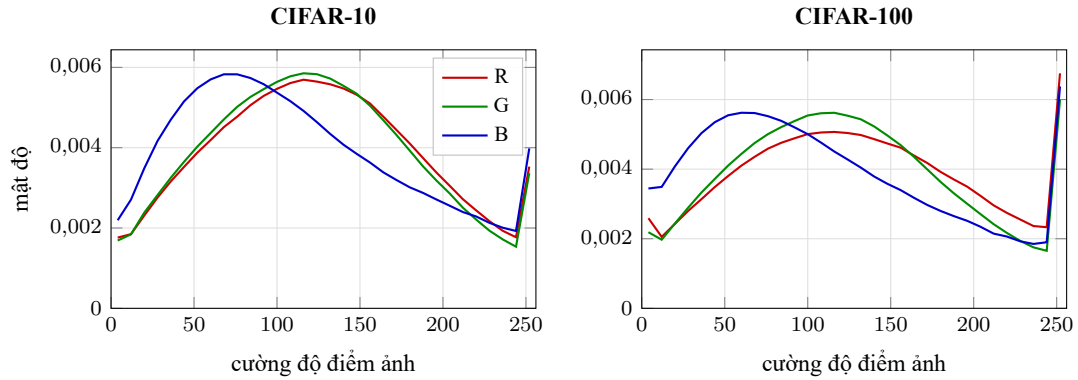
3.3 Mô tả và tiền xử lý hai tập ảnh

Chia tập. 50.000 ảnh huấn luyện được tách 45.000 ảnh train và 5.000 ảnh validation, phân tầng theo nhãn, hạt giống 42. Validation dùng để chọn checkpoint. 10.000 ảnh test chính thức chỉ được dùng một lần, ở cuối.

```
1 # a05/data.py
2 def split_train_val(y, n_val=5000, seed=SEED):
3     """Tách n_val ảnh làm validation, phân tầng theo nhãn. Trả về (idx_train, idx_val)."""
4     idx = np.arange(len(y))
5     return train_test_split(idx, test_size=n_val, stratify=y, random_state=seed)
```

Mã nguồn 26. Tách validation phân tầng

Chuẩn hoá. Mỗi kênh được trừ trung bình rồi chia độ lệch chuẩn, cả hai tính trên nhánh train. Với CIFAR-10, trung bình ba kênh R, G, B là 0,4911, 0,4821, 0,4464 và độ lệch chuẩn là 0,2469, 0,2434, 0,2616 (theo thang [0, 1]). Con số của CIFAR-100 lần lượt là 0,5071, 0,4864, 0,4406 và 0,2672, 0,2563, 0,276. Hình 3.3 cho thấy kênh xanh dương lệch về phía tối hơn hai kênh còn lại ở cả hai tập, và có đỉnh ở giá trị 255 do nền trời và nền trắng.



Hình 3.3. Phân bố cường độ điểm ảnh theo kênh trên nhánh train

Nạp dữ liệu. Hai tệp .tar.gz gốc được đọc thẳng bằng tarfile và pickle, không giải nén ra đĩa, rồi lưu đệm thành .npz.

```

1 # a05/data.py
2 def load_cifar(name):
3     """Trả về dict: x_train, y_train, x_test, y_test (ảnh uint8 NHWC), class_names;
4     riêng CIFAR-100 thêm yc_train, yc_test (20 siêu lớp) và coarse_names."""
5     cache = DATA / name / f"{name}.npz"
6     if cache.exists():
7         z = np.load(cache, allow_pickle=False)
8         return {k: (z[k].tolist() if k.endswith("names") else z[k]) for k in z.files}
9     tar_path, root = CIFAR_FILES[name]
10    with tarfile.open(DATA / tar_path) as tar:
11        if name == "cifar10":
12            train = [_read_pickle(tar, f"{root}/data_batch_{i}") for i in range(1, 6)]
13            test = _read_pickle(tar, f"{root}/test_batch")
14            meta = _read_pickle(tar, f"{root}/batches.meta")
15            out = {
16                "x_train": _to_hwc(np.concatenate([b["data"] for b in train])),
17                "y_train": np.concatenate([b["labels"] for b in train]).astype(np.int64),
18                "x_test": _to_hwc(test["data"]),
19                "y_test": np.array(test["labels"], dtype=np.int64),
20                "class_names": meta["label_names"],
21            }
22        else:
23            train = _read_pickle(tar, f"{root}/train")
24            test = _read_pickle(tar, f"{root}/test")
25            meta = _read_pickle(tar, f"{root}/meta")
26            out = {
27                "x_train": _to_hwc(train["data"]),
28                "y_train": np.array(train["fine_labels"], dtype=np.int64),
29                "yc_train": np.array(train["coarse_labels"], dtype=np.int64),
30                "x_test": _to_hwc(test["data"]),
31                "y_test": np.array(test["fine_labels"], dtype=np.int64),
32                "yc_test": np.array(test["coarse_labels"], dtype=np.int64),
33                "class_names": meta["fine_label_names"],
34                "coarse_names": meta["coarse_label_names"],
35            }
36    np.savez(cache, **out)
37    return out

```

Mã nguồn 27. Đọc CIFAR-10/100 từ tệp gốc

Dữ liệu nằm sẵn trên GPU. Trên Windows, DataLoader nhiều tiến trình vừa chậm vừa dễ lỗi. Ở dạng uint8, cả tập ảnh đủ nhỏ để nằm gọn trong VRAM, nên được đưa hết lên GPU một lần. Mỗi lô được cắt ra bằng chỉ số ngẫu nhiên, chuẩn hoá và tăng cường ngay trên GPU. Tăng cường gồm hai phép: đệm 4 điểm ảnh rồi cắt ngẫu nhiên lại 32×32 (chịch ảnh tối đa 4 điểm theo mỗi chiều), và lật ngang với xác suất 0,5. Chỉ nhánh train được tăng cường.

```

1 # a05/data.py
2 def gpu_augment(xb, gen=None, pad=4):
3     """Crop ngẫu nhiên 32x32 từ ảnh đã đệm 0 thêm `pad` điểm ảnh, rồi Lật ngang với xác suất 0,5."""
4     B, C, H, W = xb.shape
5     dev = xb.device
6     xp = torch.nn.functional.pad(xb, (pad, pad, pad, pad))
7     oy = torch.randint(0, 2 * pad + 1, (B,), device=dev, generator=gen)
8     ox = torch.randint(0, 2 * pad + 1, (B,), device=dev, generator=gen)
9     rows = (oy[:, None] + torch.arange(H, device=dev))[:, :, None]
10    cols = (ox[:, None] + torch.arange(W, device=dev))[:, :, None, :]
11    out = xp[torch.arange(B, device=dev)[ :, None, None], :, rows, cols].permute(0, 3, 1, 2)
12    flip = torch.rand(B, device=dev, generator=gen) < 0.5
13    out[flip] = out[flip].flip(3)
14    return out

```

Mã nguồn 28. Tăng cường dữ liệu chạy trên GPU

```

1 # a05/data.py
2 def batches(self, split, batch_size, augment=False, shuffle=False, gen=None):
3     x, y = self.x[split], self.y[split]
4     n = len(y)
5     order = torch.randperm(n, device=self.device, generator=gen) if shuffle else torch.arange(n, device=self.device)
6     for s in range(0, n, batch_size):
7         idx = order[s:s + batch_size]
8         xb = x[idx]
9         if augment:
10            xb = gpu_augment(xb, gen)
11        yield self.normalize(xb), y[idx]

```

Mã nguồn 29. Sinh lô từ tensor nằm sẵn trên GPU

Dạng tensor. Mỗi lô đưa vào mạng có dạng $[B, 3, 32, 32]$ kiểu float32, $B = 256$.

3.4 Diabetes Prediction Dataset

3.4.1 Mô tả

Tập Diabetes Prediction [12] gồm 100.000 hồ sơ bệnh nhân, mỗi hồ sơ có 8 đặc trưng và nhãn diabetes (1 là mắc bệnh):

- Nhân khẩu: gender (Female, Male, Other) và age.
- Tiền sử: hypertension, heart_disease (0/1) và smoking_history (6 mức).
- Chỉ số cơ thể và xét nghiệm: bmi, HbA1c_level (%), blood_glucose_level (mg/dL).

Tập có 3.854 dòng trùng lặp hoàn toàn. Các dòng này bị bỏ trước khi chia tập, nếu không một bản sao có thể nằm ở train còn bản kia nằm ở test. Sau khi bỏ trùng còn 96.146 dòng, trong đó 8.482 dòng dương tính (8,82%). Mất cân bằng này quyết định cách đánh giá ở Chương 5: trên tập test, đoán mọi người đều khoẻ đã đạt accuracy 91,18%, nên accuracy một mình không nói được gì.

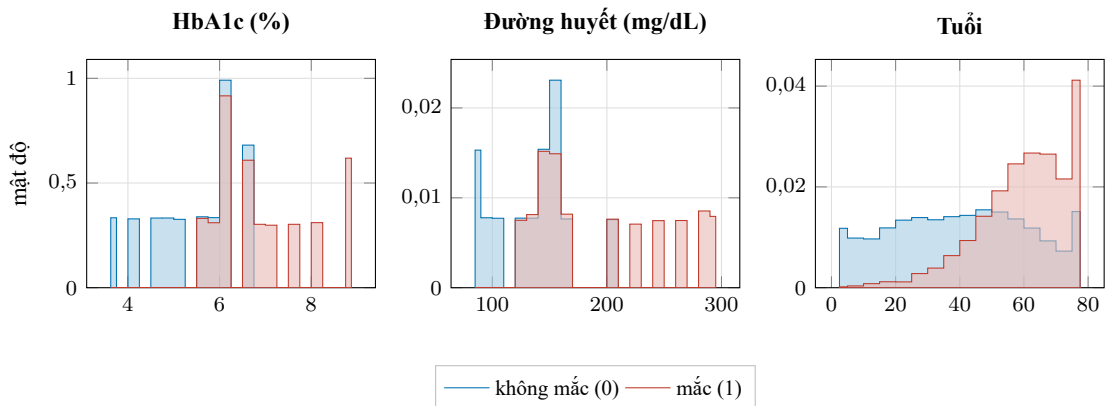
Bảng 3.2. Thống kê mô tả bốn đặc trưng số (sau khi bỏ trùng)

Đặc trưng	Trung bình	Độ lệch chuẩn	Nhỏ nhất	Lớn nhất
age	41,79	22,46	0,08	80,00
bmi	27,32	6,77	10,01	95,69
HbA1c_level	5,53	1,07	3,50	9,00
blood_glucose_level	138,22	40,91	80,00	300,00

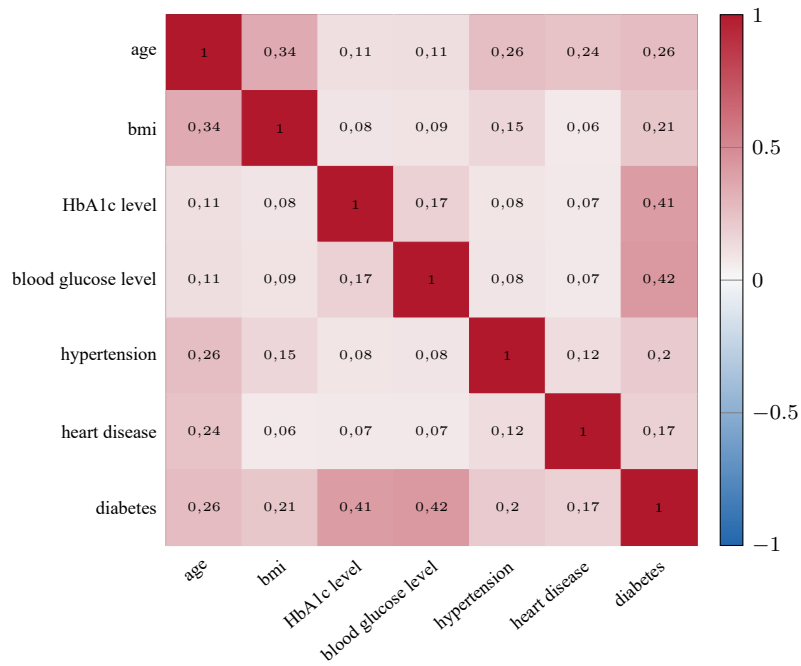
Giá trị "No Info" của smoking_history chiếm 34,21 % số dòng. Đây không phải giá trị thiếu ngẫu nhiên mà là một nhóm bệnh nhân không khai báo, nên được giữ lại như một mức riêng.

3.4.2 Quan hệ giữa đặc trưng và nhãn

Hình 3.4 tách phân bố của ba đặc trưng theo nhãn. Hai chỉ số xét nghiệm tách hai lớp rõ nhất. Notebook 02 tìm được hai ngưỡng mà phía trên đó *mọi* bệnh nhân đều mắc bệnh: HbA1c từ 6,8 % và đường huyết từ 220 mg/dL. Có 66,86 % ca dương tính vượt ít nhất một trong hai ngưỡng này, nên phần lớn ca dương rất dễ nhận ra; phần khó của bài toán là số ca còn lại, có chỉ số xét nghiệm nằm trong vùng chồng lấn với người khỏe. Histogram của hai chỉ số này có dạng răng cưa vì trong tập dữ liệu chúng chỉ nhận một số ít giá trị rời rạc, không phải số đo liên tục. Ma trận tương quan (Hình 3.5) cho kết quả khớp: hệ số tương quan với nhãn cao nhất là của blood_glucose_level (0,4243) và HbA1c_level (0,4064), còn age là 0,2649.



Hình 3.4. Phân bố HbA1c, đường huyết và tuổi theo hai lớp (mật độ, sau khi bỏ trùng)



Hình 3.5. Hệ số tương quan Pearson giữa các đặc trưng số, nhị phân và nhãn

3.4.3 Tiền xử lý và dạng tensor

Sau khi bỏ trùng, dữ liệu được chia 70/15/15 phân tầng theo nhãn thành 67.302 dòng train, 14.422 dòng validation và 14.422 dòng test. Mọi phép biến đổi có tham số đều *chỉ học trên nhánh train* rồi áp dụng nguyên cho hai nhánh còn lại: StandardScaler cho bốn cột số, và danh mục one-hot cho gender và smoking_history. Kết quả là 15 đặc trưng.

```

1 # a05/data.py
2 def prepare_diabetes(seed=SEED):
3     """Trả về dict X/y cho train/val/test (float32) và tên cột sau mã hoá.
4
5     Scaler và danh mục one-hot chỉ học từ nhánh train để không rò rỉ thông tin.
6     """
7     df = load_diabetes_raw().drop_duplicates().reset_index(drop=True)
8     y = df[DIAB_TARGET].to_numpy()
9     idx = np.arange(len(df))
10    tr, rest = train_test_split(idx, test_size=0.30, stratify=y, random_state=seed)
11    va, te = train_test_split(rest, test_size=0.50, stratify=y[rest], random_state=seed)
12
13    cats = {c: sorted(df.loc[tr, c].unique()) for c in DIAB_CATEGORICAL}
14    scaler = StandardScaler().fit(df.loc[tr, DIAB_NUMERIC])
15
16    def encode(rows):
17        part = df.loc[rows]
18        cols = [scaler.transform(part[DIAB_NUMERIC]), part[DIAB_BINARY].to_numpy(float)]
19        for c in DIAB_CATEGORICAL:
20            cols.append(np.stack([(part[c] == v).to_numpy(float) for v in cats[c]], axis=1))
21        return np.concatenate(cols, axis=1).astype(np.float32)
22
23    names = DIAB_NUMERIC + DIAB_BINARY + [f"{c}={v}" for c in DIAB_CATEGORICAL for v in cats[c]]
24    return {
25        "X": {"train": encode(tr), "val": encode(va), "test": encode(te)},
26        "y": {"train": y[tr].astype(np.float32), "val": y[va].astype(np.float32), "test": y[te].astype(np.float32)},
27        "feature_names": names,
28        "n_duplicates": int(len(load_diabetes_raw()) - len(df)),
29        "n_rows": int(len(df)),
30    }

```

Mã nguồn 30. Chuẩn bị dữ liệu Diabetes, không rò rỉ từ test

Mỗi bệnh nhân là một vector $\mathbf{x} \in \mathbb{R}^{15}$, được đưa vào mạng dưới dạng $[B, 1, 15]$: một kênh, chiều dài bằng số đặc trưng. Bộ lọc Conv1D kích thước 3 trượt dọc các cột đặc trưng kề nhau. Thứ tự cột là cố định (số, nhị phân, rồi one-hot) nhưng không mang ý nghĩa không gian như điểm ảnh. Chương 5 sẽ cho thấy điều này ảnh hưởng thế nào đến lợi ích của các cơ chế kiến trúc.

Chương 4

Một CNN cơ bản và ba mô hình phát triển

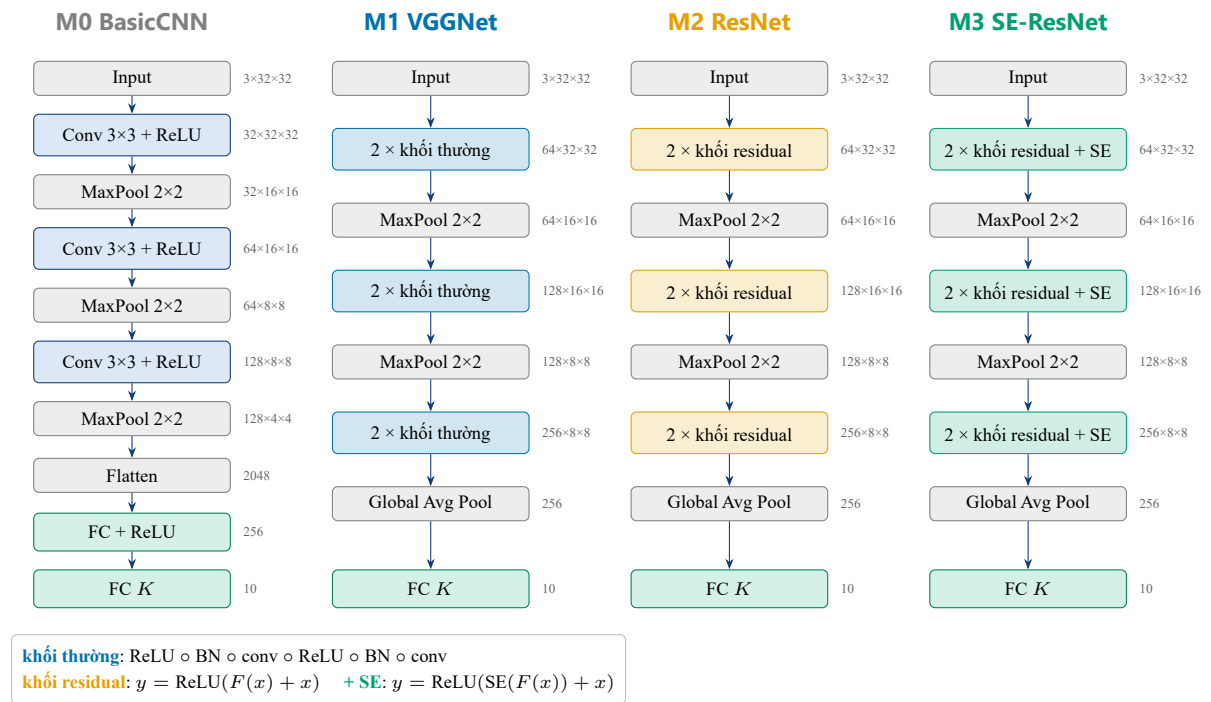
Chương này trả lời mục 4 của đề: xây một CNN cơ bản và ba mô hình phát triển, rồi huấn luyện cả bốn trên ba tập dữ liệu của Chương 3. Cả bốn mô hình được viết từ đầu bằng PyTorch [13], không dùng trọng số huấn luyện sẵn.

4.1 Nguyên tắc thiết kế: mỗi bước thêm đúng một cơ chế

Nếu bốn mô hình khác nhau ở nhiều chỗ cùng lúc thì không thể biết chênh lệch kết quả đến từ đâu. Vì vậy mỗi mô hình được dựng từ mô hình trước bằng cách thêm *một* cơ chế ở Chương 2:

- **M0 BasicCNN**: khuôn LeNet/AlexNet, ba lần lặp Conv–ReLU–MaxPool rồi hai tầng nối đầy đủ. Không BatchNorm, không đường tắt.
- **M1 VGGNet**: ba tầng, mỗi tầng hai khối, mỗi khối là hai conv 3×3 có BatchNorm [14]; cuối mạng là Global Average Pooling thay cho các tầng nối đầy đủ lớn.
- **M2 ResNet**: đúng khung của M1, mỗi khối thêm đường tắt $y = \text{ReLU}(F(x) + x)$. Khi số kênh đổi, đường tắt đi qua một conv 1×1 .
- **M3 SE-ResNet**: đúng M2, thêm khối SE trên nhánh F trước phép cộng.

Bước M0→M1 gộp ba thay đổi (sâu hơn, BatchNorm, GAP), vì đó chính là bước chuyển từ AlexNet sang VGG trong lịch sử. Hai bước sau thì mỗi bước chỉ khác đúng một cơ chế: M1 và M2 chỉ khác nhau ở đường tắt, M2 và M3 chỉ khác nhau ở khối SE. Hình 4.1 đặt bốn mô hình cạnh nhau. Shape ghi bên phải mỗi khối được đọc từ một lượt forward thật trong notebook 06, không phải tính tay.



Hình 4.1. Bốn mô hình cho ảnh $32 \times 32 \times 3$; M1, M2, M3 dùng chung khung và chỉ khác nhau ở nội dung của khối

4.2 Mã nguồn bốn mô hình

M0 là một lớp riêng. M1, M2, M3 dùng chung lớp StagedCNN, chỉ khác hai cờ residual và se của lớp Block. Nhờ vậy “chỉ khác một cơ chế” được đảm bảo ngay trong mã nguồn: hai mô hình liền kề chạy qua cùng một đoạn mã, chỉ khác một nhánh if.

```

1 # a05/models.py
2 class BasicCNN(nn.Module):
3     """M0: mạng gốc kiểu LeNet/AlexNet, không BatchNorm, không đường tắt."""
4
5     def __init__(self, dim, in_shape, n_out, widths, hidden):
6         super().__init__()
7         Conv, _, Pool, _ = _ops(dim)
8         layers, c = [], in_shape[0]
9         for w in widths:
10             layers += [Conv(c, w, 3, padding=1), nn.ReLU(), Pool(2)]
11             c = w
12         self.features = nn.Sequential(*layers)
13         with torch.no_grad():
14             n_flat = self.features(torch.zeros(1, *in_shape)).numel()
15         self.head = nn.Sequential(nn.Flatten(), nn.Linear(n_flat, hidden), nn.ReLU(), nn.Linear(hidden, n_out))
16
17     def forward(self, x):
18         return self.head(self.features(x))

```

Mã nguồn 31. M0: CNN cơ bản

```

1 # a05/models.py
2 class Block(nn.Module):
3     """Khối hai conv 3x3. residual=False là khối VGG, True là BasicBlock của ResNet,
4     se=True thêm chú ý theo kênh vào nhánh F."""
5
6     def __init__(self, dim, c_in, c_out, residual, se):
7         super().__init__()
8         Conv, BN, _, _ = _ops(dim)
9         self.F = nn.Sequential(
10             Conv(c_in, c_out, 3, padding=1, bias=False), BN(c_out), nn.ReLU(),
11             Conv(c_out, c_out, 3, padding=1, bias=False), BN(c_out))
12         self.se = SEBlock(c_out) if se else nn.Identity()
13         self.residual = residual
14         self.shortcut = nn.Identity()
15         if residual and c_in != c_out:
16             self.shortcut = nn.Sequential(Conv(c_in, c_out, 1, bias=False), BN(c_out))
17
18     def forward(self, x):
19         out = self.se(self.F(x))
20         if self.residual:
21             out = out + self.shortcut(x)
22         return torch.relu(out)

```

Mã nguồn 32. Khối hai conv 3×3 , bật tắt đường tắt và SE bằng cờ

```

1 # a05/models.py
2 class StagedCNN(nn.Module):
3     """M1/M2/M3: cùng khung 3 tầng x 2 khối, chỉ khác hai cờ residual và se."""
4
5     def __init__(self, dim, in_shape, n_out, widths, residual, se):
6         super().__init__()
7         _, _, Pool, GAP = _ops(dim)
8         stages, c = [], in_shape[0]
9         for i, w in enumerate(widths):
10             stages += [Block(dim, c, w, residual, se), Block(dim, w, w, residual, se)]
11             if i < len(widths) - 1:
12                 stages.append(Pool(2))
13             c = w
14         self.features = nn.Sequential(*stages)
15         self.head = nn.Sequential(GAP(1), nn.Flatten(), nn.Linear(c, n_out))
16
17     def forward(self, x):
18         return self.head(self.features(x))

```

Mã nguồn 33. Khung chung của M1, M2, M3

Hàm `build_model` dựng mô hình theo tên và theo số chiều. Tham số `dim` chọn giữa bộ (Conv2d, BatchNorm2d, MaxPool2d) và bộ 1D tương ứng, nên cùng một định nghĩa phục vụ cả ảnh lẫn dữ liệu bảng. Mọi tầng tích chập được khởi tạo He (`kaiming_normal_`), phù hợp với ReLU.

```

1 # a05/models.py
2 def build_model(name, dim, in_shape, n_out):
3     """Dựng mô hình theo tên. dim=2: ảnh (C, H, W); dim=1: bảng (1, số đặc trưng)."""
4     basic_w, staged_w = ((32, 64, 128), (64, 128, 256)) if dim == 2 else ((16, 32, 64), (16, 32, 64))
5     hidden = 256 if dim == 2 else 64
6     if name == "basic":
7         m = BasicCNN(dim, in_shape, n_out, basic_w, hidden)
8     elif name in ("vgg", "resnet", "seresnet"):
9         m = StagedCNN(dim, in_shape, n_out, staged_w, residual=name != "vgg", se=name == "seresnet")
10    else:
11        raise ValueError(f"Không có mô hình '{name}', chọn một trong {MODEL_NAMES}")
12    for mod in m.modules():
13        if isinstance(mod, (nn.Conv1d, nn.Conv2d)):
14            nn.init.kaiming_normal_(mod.weight, nonlinearity="relu")
15    return m

```

Mã nguồn 34. Dựng mô hình theo tên, cho cả 2D và 1D

4.3 Dò shape tầng theo tầng

Bảng 4.1 là đầu ra thật của từng tầng con khi một tensor $1 \times 3 \times 32 \times 32$ đi qua M0. Ba lần MaxPool đưa 32×32 về 4×4 ; tầng Flatten nối $128 \cdot 4 \cdot 4$ giá trị vào tầng nối đầy đủ, và riêng tầng này chiếm phần lớn số tham số của M0. Ở M1, M2, M3, Global Average Pooling nén mỗi kênh về một số, nên tầng nối đầy đủ cuối chỉ còn $256 \times K$ trọng số (Bảng 4.2).

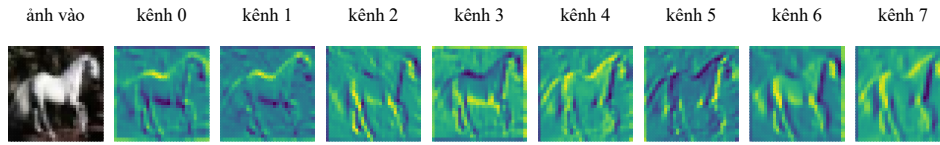
Bảng 4.1. Đồ shape của M0 (2D, $K = 10$)

#	Tầng	Shape đầu ra	Tham số
0	Input	$3 \times 32 \times 32$	–
1	Conv2d	$32 \times 32 \times 32$	896
2	ReLU	$32 \times 32 \times 32$	–
3	MaxPool2d	$32 \times 16 \times 16$	–
4	Conv2d	$64 \times 16 \times 16$	18.496
5	ReLU	$64 \times 16 \times 16$	–
6	MaxPool2d	$64 \times 8 \times 8$	–
7	Conv2d	$128 \times 8 \times 8$	73.856
8	ReLU	$128 \times 8 \times 8$	–
9	MaxPool2d	$128 \times 4 \times 4$	–
10	Flatten	2048	–
11	Linear	256	524.544
12	ReLU	256	–
13	Linear	10	2.570

Bảng 4.2. Đồ shape của M3 (2D, $K = 10$)

#	Tầng	Shape đầu ra	Tham số
0	Input	$3 \times 32 \times 32$	–
1	Block	$64 \times 32 \times 32$	39.748
2	Block	$64 \times 32 \times 32$	74.564
3	MaxPool2d	$64 \times 16 \times 16$	–
4	Block	$128 \times 16 \times 16$	232.328
5	Block	$128 \times 16 \times 16$	297.608
6	MaxPool2d	$128 \times 8 \times 8$	–
7	Block	$256 \times 8 \times 8$	927.504
8	Block	$256 \times 8 \times 8$	1.189.136
9	AdaptiveAvgPool2d	$256 \times 1 \times 1$	–
10	Flatten	256	–
11	Linear	10	2.570

Hình 4.2 cho thấy tầng tích chập đầu của M0 (sau khi huấn luyện trên CIFAR-10) làm gì với một ảnh test: mỗi kênh là ảnh vào sau khi qua một bộ lọc $3 \times 3 \times 3$. Có kênh làm nổi biên theo một hướng, có kênh phản ứng với vùng sáng hoặc với một màu. Đây là đầu vào của các tầng sau.



Hình 4.2. Ảnh test lớp horse và 8 feature map đầu tiên sau tầng tích chập đầu của M0

4.4 Bản 1D cho Diabetes

Với Diabetes, `build_model(name, dim=1, ...)` giữ nguyên bốn cơ chế, chỉ đổi mọi tầng 2D sang 1D, giảm độ rộng xuống 16/32/64 kênh và cho ra một logit. Đầu vào là `[B, 1, 15]`. Bảng 4.3 là shape của M0 bản 1D; ba lần MaxPool làm chiều dài giảm từ 15 xuống 1.

Bảng 4.3. Dò shape của M0 bản 1D cho Diabetes

#	Tầng	Shape đầu ra	Tham số
0	Input	1×15	–
1	Conv1d	16×15	64
2	ReLU	16×15	–
3	MaxPool1d	16×7	–
4	Conv1d	32×7	1.568
5	ReLU	32×7	–
6	MaxPool1d	32×3	–
7	Conv1d	64×3	6.208
8	ReLU	64×3	–
9	MaxPool1d	64×1	–
10	Flatten	64	–
11	Linear	64	4.160
12	ReLU	64	–
13	Linear	1	65

4.5 Huấn luyện

4.5.1 Cấu hình

Trong cùng một tập, bốn mô hình dùng đúng một cấu hình (Bảng 4.4) và cùng hạt giống 42. Mỗi mô hình chạy một lần. Checkpoint được giữ ở epoch có loss validation nhỏ nhất và được nạp lại trước khi đánh giá trên tập test.

Bảng 4.4. Cấu hình huấn luyện

	CIFAR-10 và CIFAR-100	Diabetes
Thiết bị	GPU NVIDIA GeForce RTX 4060 Laptop GPU, AMP fp16	CPU, 8 luồng
Tối ưu	SGD nesterov, momentum 0,9	AdamW
Tốc độ học	OneCycle, đỉnh 0,1	cố định 0,001
Weight decay	$5,00 \times 10^{-4}$	$1,00 \times 10^{-4}$
Epoch, cỡ lô	30, 256	tối đa 30, 512
Dừng sớm	không	sau 5 epoch không giảm val loss
Cắt gradient	$\ \nabla\ _2 \leq 2$	không
Hàm mất mát	cross-entropy	BCE, pos_weight = số âm / số dương
Ngưỡng phân loại	argmax	chọn trên validation theo F1

Lịch OneCycle [15] tăng tốc độ học từ nhỏ lên đỉnh trong 30 % số bước đầu, rồi giảm dần về gần 0. Pha giảm cuối giúp mô hình hội tụ vào một điểm tốt trong 30 epoch mà không cần dò lịch giảm bằng tay.

4.5.2 Vòng lặp huấn luyện

Một epoch là một lượt qua dữ liệu: với mỗi lô, tính $L(f_\theta(x), y)$, lan truyền ngược, rồi cập nhật θ . Hàm `train_one_epoch` dùng chung cho cả đánh giá: khi `opt=None`, nó chỉ tính loss và accuracy. Với AMP, phép tính chạy ở fp16, còn `GradScaler` nhân loss lên trước khi lan truyền ngược để gradient nhỏ không bị làm tròn về 0.

```

1 # a05/train.py
2 def train_one_epoch(model, batches, loss_fn, opt=None, sched=None, scaler=None, amp=False, clip=None):
3     """Chạy một lượt qua dữ liệu. opt=None nghĩa là chỉ đánh giá, không cập nhật.
4     clip: giới hạn chuẩn L2 của gradient (None = không cắt). Trả về (loss trung bình, accuracy)."""
5     training = opt is not None
6     model.train(training)
7     tot_loss, tot_ok, n = 0.0, 0, 0
8     with torch.set_grad_enabled(training):
9         for xb, yb in batches:
10             with torch.autocast(device_type=xb.device.type, dtype=torch.float16, enabled=amp):
11                 out = model(xb)
12                 loss = loss_fn(out, yb)
13                 if training:
14                     opt.zero_grad(set_to_none=True)
15                     if scaler is not None:
16                         scaler.scale(loss).backward()
17                         scaler.unscale_(opt)
18                     else:
19                         loss.backward()
20                     if clip is not None:
21                         torch.nn.utils.clip_grad_norm_(model.parameters(), clip)
22                     if scaler is not None:
23                         scaler.step(opt)
24                         scaler.update()
25                     else:
26                         opt.step()
27                     if sched is not None:
28                         sched.step()
29                 tot_loss = tot_loss + loss.detach().float() * len(yb)
30                 tot_ok = tot_ok + _n_correct(out.detach().float(), yb)
31                 n += len(yb)
32     return float(tot_loss) / n, float(tot_ok) / n

```

Mã nguồn 35. Một epoch huấn luyện hoặc đánh giá

```

1 # a05/train.py
2 def fit(model, batches, loss_fn, opt, epochs, device, sched=None, amp=False, patience=None, clip=None,
3         log=print):
4     """Huấn luyện `epochs` epoch, giữ trạng thái có val Loss nhỏ nhất.
5
6     batches(split) phải trả về một iterator Lô mới cho 'train' hoặc 'val'.
7     patience: dừng sớm nếu val Loss không giảm sau chừng ấy epoch (None = không dừng).
8     Trả về dict history, best_epoch, best_state (trên CPU), epoch_seconds.
9     """
10    scaler = torch.amp.GradScaler("cuda") if amp else None
11    history, best, best_state, best_epoch, wait = [], float("inf"), None, 0, 0
12    for ep in range(1, epochs + 1):
13        _sync(device)
14        t0 = time.perf_counter()
15        tr_loss, tr_acc = train_one_epoch(model, batches("train"), loss_fn, opt, sched, scaler, amp, clip)
16        _sync(device)
17        secs = time.perf_counter() - t0
18        va_loss, va_acc = train_one_epoch(model, batches("val"), loss_fn, amp=amp)
19        history.append({"epoch": ep, "train_loss": tr_loss, "train_acc": tr_acc,
20                      "val_loss": va_loss, "val_acc": va_acc, "seconds": secs})
21        log(f" epoch {ep:2d} train {tr_loss:.4f}/{tr_acc:.4f} val {va_loss:.4f}/{va_acc:.4f} {secs:.1f}s")
22        if va_loss < best:
23            best, best_epoch, wait = va_loss, ep, 0
24            best_state = {k: v.detach().cpu().clone() for k, v in model.state_dict().items()}
25        else:
26            wait += 1
27            if patience is not None and wait >= patience:
28                log(f" dừng sớm ở epoch {ep}, tốt nhất là epoch {best_epoch}")
29                break
30    model.load_state_dict(best_state)
31    return {"history": history, "best_epoch": best_epoch, "best_state": best_state,
32          "epoch_seconds": float(np.mean([h["seconds"] for h in history]))}

```

Mã nguồn 36. Huấn luyện nhiều epoch, giữ checkpoint theo val loss

Hai hàm dưới đây gói phần riêng của từng loại dữ liệu: bộ tối ưu, lịch tốc độ học, hàm mất mát và cách sinh lô.

```

1 # a05/train.py
2 def train_image_model(name, data, cfg=IMAGE_CFG, seed=42, log=print):
3     """Huấn luyện một mô hình 2D trên CIFAR: SGD nesterov + OneCycle, AMP fp16 trên GPU.
4
5     Cài chuẩn gradient ở cả bốn mô hình: BasicCNN không có BatchNorm nên phân kỳ
6     khi lr lên đỉnh 0,1 nếu không cắt (outputs/figdata/cifar10_basic_noclip_history.dat)."""
7     set_seed(seed)
8     dev = data.device
9     model = build_model(name, 2, (3, 32, 32), data.n_classes).to(dev)
10    opt = torch.optim.SGD(model.parameters(), lr=cfg["max_lr"], momentum=cfg["momentum"],
11                          nesterov=True, weight_decay=cfg["weight_decay"])
12    steps = -(-len(data.y["train"]) // cfg["batch_size"])
13    sched = torch.optim.lr_scheduler.OneCycleLR(opt, cfg["max_lr"], epochs=cfg["epochs"], steps_per_epoch=steps)
14    gen = torch.Generator(device=dev)
15    gen.manual_seed(seed)
16    batches = lambda split: data.batches(split, cfg["batch_size"], augment=split == "train",
17                                         shuffle=split == "train", gen=gen)
18    res = fit(model, batches, nn.CrossEntropyLoss(), opt, cfg["epochs"], dev, sched, amp=True,
19             clip=cfg["grad_clip"], log=log)
20    return model, res

```

Mã nguồn 37. Huấn luyện một mô hình 2D trên CIFAR

```

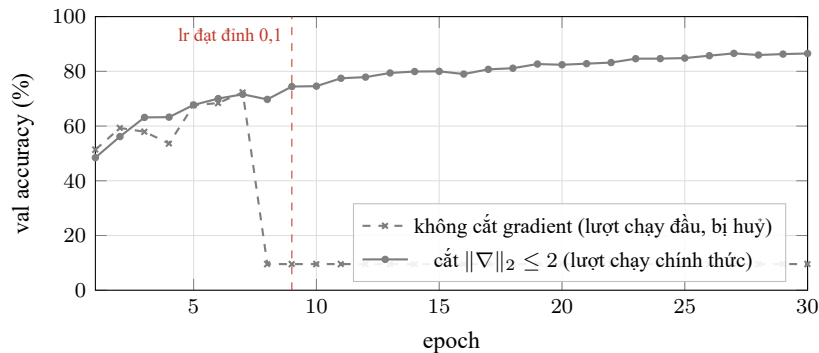
1 # a05/train.py
2 def train_tabular_model(name, D, cfg=TAB_CFG, seed=42, log=print):
3     """Huấn luyện một mô hình 1D trên Diabetes bằng CPU: AdamW, BCE có pos_weight, dừng sớm."""
4     set_seed(seed)
5     torch.set_num_threads(cfg["threads"])
6     model = build_model(name, 1, (1, D["X"]["train"].shape[1]), 1)
7     y_tr = D["y"]["train"]
8     pos_weight = torch.tensor((y_tr == 0).sum() / (y_tr == 1).sum())
9     bce = nn.BCEWithLogitsLoss(pos_weight=pos_weight)
10    loss_fn = lambda out, y: bce(out.reshape(-1), y)
11    opt = torch.optim.AdamW(model.parameters(), lr=cfg["lr"], weight_decay=cfg["weight_decay"])
12    gen = torch.Generator().manual_seed(seed)
13    batches = lambda split: tabular_batches(D["X"][split], D["y"][split], cfg["batch_size"],
14                                           shuffle=split == "train", gen=gen)
15    res = fit(model, batches, loss_fn, opt, cfg["epochs"], "cpu", patience=cfg["patience"], log=log)
16    return model, res

```

Mã nguồn 38. Huấn luyện một mô hình 1D trên Diabetes

4.5.3 Sự cố phân kỳ của M0 và vai trò của BatchNorm

Lượt chạy đầu tiên của M0 trên CIFAR-10 chưa cắt gradient và đã phân kỳ: loss trở thành NaN ở epoch 8, đúng lúc lịch OneCycle đưa tốc độ học lên đỉnh 0,1 (Hình 4.3). Giải thích hợp lý nhất là M0 thiếu BatchNorm: BatchNorm chuẩn hoá đầu ra mỗi tầng về trung bình 0 và phương sai 1, nên độ lớn của tín hiệu và của gradient không tăng dần qua các tầng, và vì thế chịu được tốc độ học lớn hơn [14]. Đây là suy luận từ cơ chế, không phải kết quả của một thí nghiệm tách riêng.



Hình 4.3. M0 trên CIFAR-10: không cắt gradient thì phân kỳ khi tốc độ học lên đỉnh

Để bốn mô hình vẫn dùng chung một cấu hình, lượt chạy chính thức bật cắt chuẩn gradient cho *cả bốn*. Bài không chạy riêng M1, M2, M3 khi tắt cắt gradient, cũng không đo ngưỡng bị chạm bao nhiêu lần, nên không kết luận được việc cắt có ảnh hưởng tới ba mô hình này hay không. Lượt phân kỳ được giữ lại trong `cifar10_basic_noclip_history.dat` làm bằng chứng.

4.5.4 Một lượt thí nghiệm

Notebook 03, 04, 05 đều chỉ gọi một hàm: huấn luyện đủ bốn mô hình, lưu checkpoint và metadata.json, ghi đường học ra .dat, rồi đo chất lượng và chi phí trên tập test. Ba tập vì thế đi qua đúng cùng một quy trình.

```

1 # a05/experiment.py
2 def run_image_experiment(ds_name, data, log=print):
3     """Huấn luyện bốn mô hình 2D trên một tập CIFAR đã nạp lên GPU; trả về dict số liệu."""
4     dev = data.device
5     results = {}
6     for name in MODEL_NAMES:
7         log(f"== {ds_name} / {name}")
8         model, res = train_image_model(name, data, log=log)
9         tag = f"{ds_name}_{name}"
10        _history_dat(tag, res["history"])
11        logits, y = predict(model, data.batches("test", 1000), amp=True)
12        x_probe = data.normalize(data.x["test"][:256])
13        with torch.autocast("cuda", dtype=torch.float16):
14            infer_ms = M.time_inference(model, x_probe)
15        r = M.classification_report(logits, y)
16        r.update(params=M.count_params(model), macs=M.count_macs(model, x_probe),
17                epoch_s=res["epoch_seconds"], infer_ms=infer_ms, best_epoch=res["best_epoch"],
18                epochs_run=len(res["history"]),
19                best_val_acc=max(h["val_acc"] for h in res["history"]))
20        results[name] = r
21        save_checkpoint(tag, res["best_state"], {"dataset": ds_name, "model": name, "config": IMAGE_CFG,
22                                                "device": torch.cuda.get_device_name(dev), **r})
23        log(f" test acc {r['acc']:.4f} top5 {r['top5']:.4f} macro-F1 {r['macro_f1']:.4f}")
24        save_metrics(ds_name, {"models": results, "config": IMAGE_CFG,
25                              "device": torch.cuda.get_device_name(dev),
26                              "n_train": len(data.y["train"]), "n_val": len(data.y["val"]),
27                              "n_test": len(data.y["test"])}))
28    return results

```

Mã nguồn 39. Một lượt thí nghiệm đầy đủ trên một tập ảnh

Ghi chú về thời gian. Máy dùng GPU laptop NVIDIA GeForce RTX 4060 Laptop GPU. Trong suốt các lượt huấn luyện ảnh, GPU chạy nóng và tự hạ xung nhịp vì nhiệt (nvidia-smi báo cờ *SW thermal slowdown*). Số giây mỗi epoch ở Chương 5 vì vậy chậm hơn nhiều so với lúc máy còn mát. Chúng vẫn so sánh được với nhau vì cả tám lượt chạy trong cùng điều kiện, nhưng không nên đọc như thời gian chuẩn của loại GPU này.

Chương 5

So sánh và đánh giá

5.1 Tiêu chí đánh giá

Accuracy một mình không đủ để so sánh các mô hình: nó bỏ qua giá phải trả, và gần như vô nghĩa khi dữ liệu mất cân bằng như Diabetes. Chương này so sánh bốn mô hình theo năm nhóm tiêu chí:

1. **Chất lượng** trên tập test: accuracy và macro-F1 với ảnh; thêm top-5 với CIFAR-100 (100 lớp, nhiều lớp gần giống nhau); precision, recall, F1, ROC-AUC và PR-AUC với Diabetes.
2. **Chi phí**: số tham số, số phép nhân-cộng (MACs) cho một mẫu, thời gian một epoch, thời gian suy luận một mẫu.
3. **Hội tụ**: đường loss và accuracy theo epoch, epoch có val loss nhỏ nhất.
4. **Lỗi**: ma trận nhầm lẫn của mô hình tốt nhất.
5. **Biểu diễn**: bộ lọc lớp đầu và feature map của cùng một ảnh qua bốn mô hình.

Macro-F1 là trung bình F1 của từng lớp, nên một lớp bị bỏ quên sẽ kéo điểm này xuống dù accuracy vẫn cao. Top- k accuracy tính tỉ lệ mẫu có nhãn đúng nằm trong k lớp có điểm cao nhất.

```
1 # a05/metrics.py
2 def topk_accuracy(logits, y, k=1):
3     """Tỉ lệ mẫu có nhãn đúng nằm trong k lớp có điểm cao nhất."""
4     topk = np.argsort(-logits, axis=1)[: , :k]
5     return float((topk == y[:, None]).any(axis=1).mean())
```

Mã nguồn 40. Top- k accuracy

```
1 # a05/metrics.py
2 def classification_report(logits, y):
3     pred = logits.argmax(1)
4     return {"acc": topk_accuracy(logits, y, 1), "top5": topk_accuracy(logits, y, 5),
5             "macro_f1": float(f1_score(y, pred, average="macro"))}
```

Mã nguồn 41. Ba chỉ số chất lượng cho ảnh

Số MACs chỉ tính các tầng tích chập và nối đầy đủ; BatchNorm, ReLU, pooling và phép cộng đường tắt nhỏ hơn vài bậc nên được bỏ qua. Thời gian suy luận được đo trên lô 256 ảnh, có đồng bộ GPU trước và sau khi đo, rồi chia cho số ảnh.

```

1 # a05/metrics.py
2 def count_macs(model, x):
3     """Đếm phép nhân-cộng (MAC) của các tầng Conv và Linear cho một mẫu.
4
5     Conv: số phân tử đầu ra × (C_in / groups × kích thước bộ lọc); Linear: in × out.
6     BatchNorm, ReLU, pooling, cộng đường tắt không tính vì nhỏ hơn vài bậc.
7     """
8     total = [0]
9
10    def hook(m, inp, out):
11        if isinstance(m, (nn.Conv1d, nn.Conv2d)):
12            k = int(np.prod(m.kernel_size)) * m.in_channels // m.groups
13            total[0] += out[0].numel() * k
14        elif isinstance(m, nn.Linear):
15            total[0] += m.in_features * m.out_features
16
17    handles = [m.register_forward_hook(hook) for m in model.modules()
18               if isinstance(m, (nn.Conv1d, nn.Conv2d, nn.Linear))]
19    model.eval()
20    with torch.no_grad():
21        model(x[:1])
22    for h in handles:
23        h.remove()
24    return total[0]

```

Mã nguồn 42. Đếm MACs bằng forward hook

```

1 # a05/metrics.py
2 @torch.no_grad()
3 def time_inference(model, x, reps=20, warmup=5):
4     """Thời gian suy luận trung bình trên một ảnh (ms), đo trên cả Lô x."""
5     model.eval()
6     sync = torch.cuda.synchronize if x.is_cuda else (lambda: None)
7     for _ in range(warmup):
8         model(x)
9     sync()
10    t0 = time.perf_counter()
11    for _ in range(reps):
12        model(x)
13    sync()
14    return (time.perf_counter() - t0) / reps / len(x) * 1000

```

Mã nguồn 43. Đo thời gian suy luận có đồng bộ GPU

Vì mỗi mô hình chỉ chạy một hạt giống, chênh lệch nhỏ giữa hai mô hình có thể chỉ là nhiễu của lần chạy đó. Chương này quy ước: chênh lệch dưới 0,5 điểm phần trăm được ghi là *nằm trong mức dao động của một lần chạy* và không dùng để kết luận mô hình nào tốt hơn.

5.2 CIFAR-10

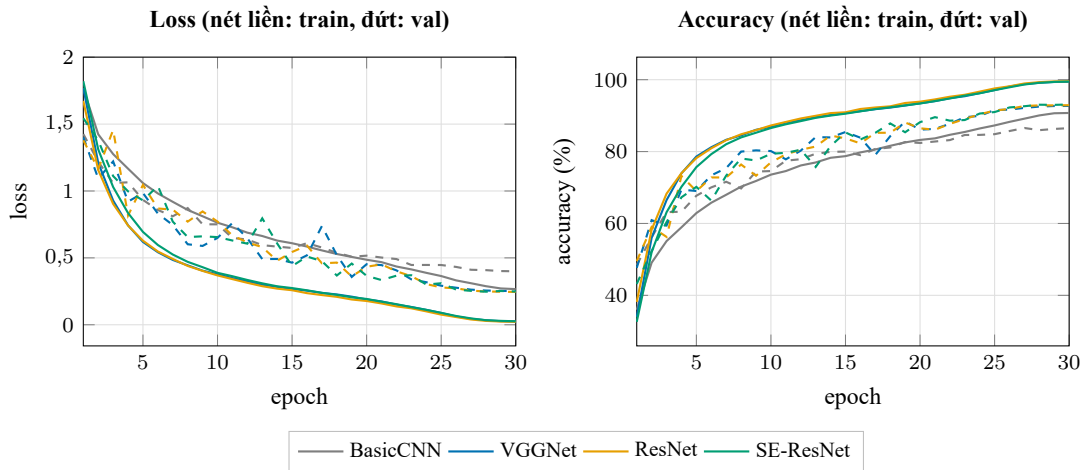
Bảng 5.1. Kết quả trên tập test CIFAR-10 (10.000 ảnh). In đậm: accuracy cao nhất

Mô hình	Acc (%)	Macro-F1 (%)	Tham số	MACs (tr.)	s/epoch	ms/ảnh	Epoch tốt
BasicCNN	85,62	85,60	620.362	10,8	2,0	0,005	30
VGGNet	92,65	92,65	2.698.954	379,3	50,1	0,099	30
ResNet	92,69	92,69	2.741.002	383,7	62,4	0,122	30
SE-ResNet	92,05	92,03	2.763.458	383,7	63,4	0,115	28

Chất lượng. Bước lớn nhất là từ M0 sang M1: accuracy tăng 7,03 điểm, từ 85,62 % lên 92,65 %. Hai bước sau không cho lợi ích nào: thêm đường tắt đôi 0,04 điểm, trong mức dao động. Thêm SE đôi $-0,64$ điểm, nhỉnh hơn ngưỡng 0,5 một chút và theo chiều xấu đi. Tuy vậy không nên đọc đó là tác hại của SE: trên tập validation, thứ tự của ba mô hình ở epoch cuối lại ngược với trên tập test, và SE-ResNet đứng đầu (92,7, 92,9 và 93,1 % cho M1, M2, M3). Macro-F1 gần như trùng accuracy vì mười lớp cân bằng.

Vì sao đường tắt không giúp gì ở đây. ResNet giải quyết bài toán *tối ưu mạng rất sâu*: khi chồng hàng chục tầng, gradient khó đi về các tầng đầu. M1 chỉ có 12 tầng tích chập, lại có BatchNorm, nên bài toán đó chưa xuất hiện. M1 đã đạt accuracy 99,42 % trên tập train ở epoch cuối, tức là tối ưu không còn là chỗ nghẽn. Đường tắt vì vậy không có gì để sửa. Kết quả này không mâu thuẫn với bài báo ResNet; nó cho thấy lợi ích của một cơ chế phụ thuộc vào việc mô hình có đang gặp đúng vấn đề mà cơ chế đó giải quyết hay không.

Hội tụ. Hình 5.1 cho thấy M0 hội tụ chậm và dừng ở accuracy train 90,74 %, trong khi ba mô hình còn lại gần như thuộc lòng tập train. Khoảng cách train-val của M0 là 4,22 điểm, của M1 là 6,72 điểm. M0 không thiếu dữ liệu mà thiếu sức chứa: nó chưa khớp hết tập train. Cả bốn mô hình đều có epoch tốt nhất ở cuối lịch huấn luyện, nhờ pha giảm tốc độ học của OneCycle.



Hình 5.1. Đường học trên CIFAR-10

5.3 CIFAR-100

Bảng 5.2. Kết quả trên tập test CIFAR-100 (10.000 ảnh, 100 lớp)

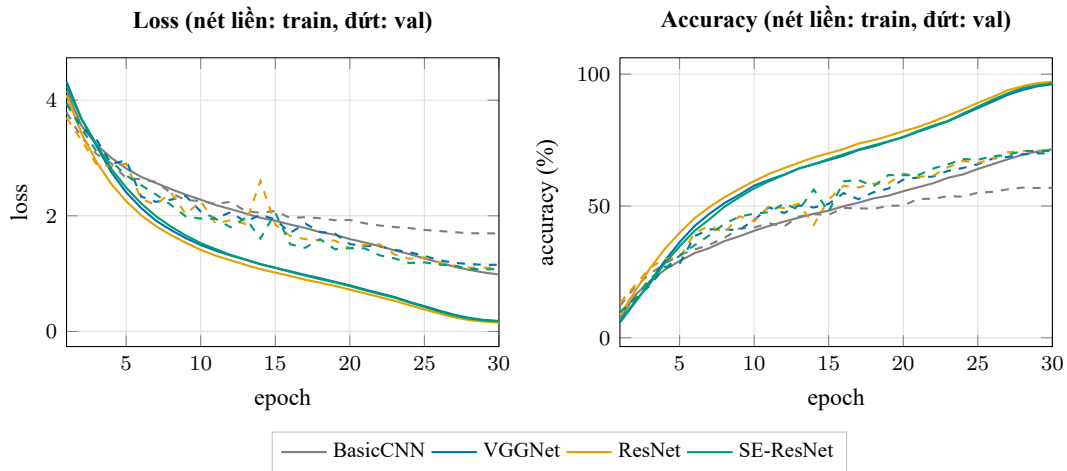
Mô hình	Acc (%)	Top-5 (%)	Macro-F1 (%)	Tham số	MACs (tr.)	s/epoch	ms/ảnh	Epoch tốt
BasicCNN	57,48	84,33	57,43	643.492	10,9	2,7	0,008	30
VGGNet	70,71	91,90	70,79	2.722.084	379,3	49,2	0,093	29
ResNet	71,71	92,26	71,71	2.764.132	383,7	58,3	0,106	29
SE-ResNet	71,86	92,56	71,92	2.786.588	383,7	59,4	0,115	29

Chất lượng. Cùng bốn kiến trúc và cùng cấu hình, chỉ tăng số lớp lên mười lần, accuracy của mọi mô hình giảm mạnh so với CIFAR-10 (Bảng 5.1 và 5.2). Bước M0→M1 lớn hơn hẳn: 13,23 điểm, từ 57,48 % lên 70,71 %. Khác với CIFAR-10, đường tắt lần này có tác dụng đo được: M2 hơn M1 1 điểm, vượt ngưỡng 0,5 điểm, và top-5 cũng tăng theo (từ 91,90 lên 92,26 %). SE vẫn chỉ thêm 0,15 điểm, trong mức dao động.

Một cách đọc hợp lý là: bài toán 100 lớp khó hơn nên M1 không còn gần như thuộc lòng tập train như ở CIFAR-10 (96,19 % so với 99,42 %), và đường tắt giúp tối ưu tốt hơn một chút. Tuy vậy, với một lần chạy duy nhất, chênh lệch một điểm chỉ đủ để nói “có dấu hiệu”, chưa đủ để kết luận chắc chắn.

Top-5. Top-5 của mô hình tốt nhất là 92,56 %, cao hơn nhiều so với top-1. Phần lớn lỗi top-1 là nhầm sang một lớp gần giống, và lớp đúng vẫn nằm trong vài lựa chọn đầu. Mục 5.6 sẽ cho thấy các lớp gần giống này thường thuộc cùng một siêu lớp.

Hội tụ. Khoảng cách train–val trên CIFAR-100 rất lớn: 26,37 điểm với M1 và 14,56 điểm với M0 (Hình 5.2). Mỗi lớp chỉ có 500 ảnh gốc, nên các mô hình có BN nhớ được tập train nhưng khái quát kém hơn nhiều so với CIFAR-10. Đây là quá khớp thật, khác với trường hợp thiếu sức chứa của M0.



Hình 5.2. Đường học trên CIFAR-100

5.4 Diabetes

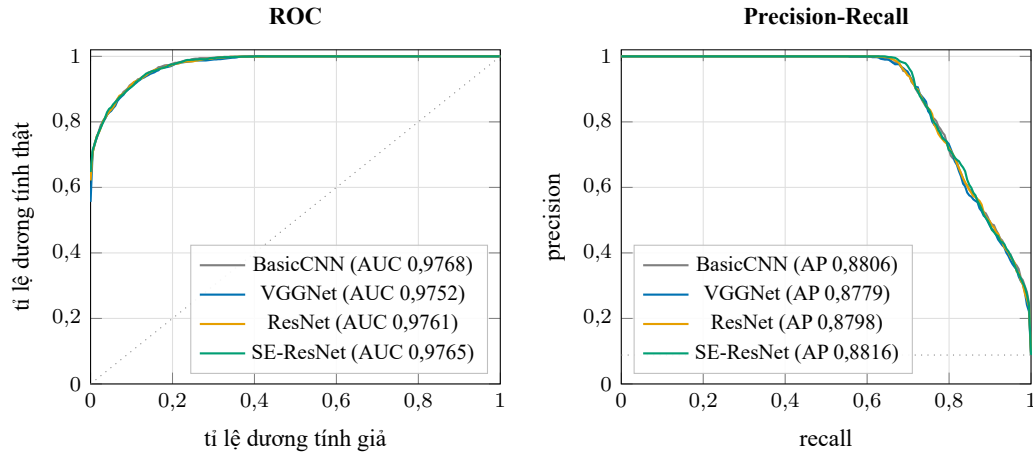
Bảng 5.3. Kết quả trên tập test Diabetes (14.422 bệnh nhân). In đậm: giá trị cao nhất mỗi cột

Mô hình	Acc	Precision	Recall	F1	ROC-AUC	PR-AUC	Ngưỡng	Tham số	s/epoch
BasicCNN	96,75	88,35	72,72	79,78	0,9768	0,8806	0,879	12.065	0,60
VGGNet	96,87	91,09	71,54	80,14	0,9752	0,8779	0,898	57.073	1,94
ResNet	96,89	91,61	71,23	80,14	0,9761	0,8798	0,869	59.873	2,15
SE-ResNet	97,12	95,34	70,83	81,28	0,9765	0,8816	0,872	61.913	2,84

Acc, Precision, Recall, F1 tính theo %, tại ngưỡng chọn trên tập validation.

Mốc so sánh. Đoán mọi bệnh nhân đều khoẻ đạt accuracy 91,18 % nhưng recall bằng 0. Accuracy của bốn mô hình từ 96,75 đến 97,12 %, chỉ hơn mốc này vài điểm. Vì vậy phần so sánh dưới đây dựa vào recall, F1 và hai diện tích dưới đường cong.

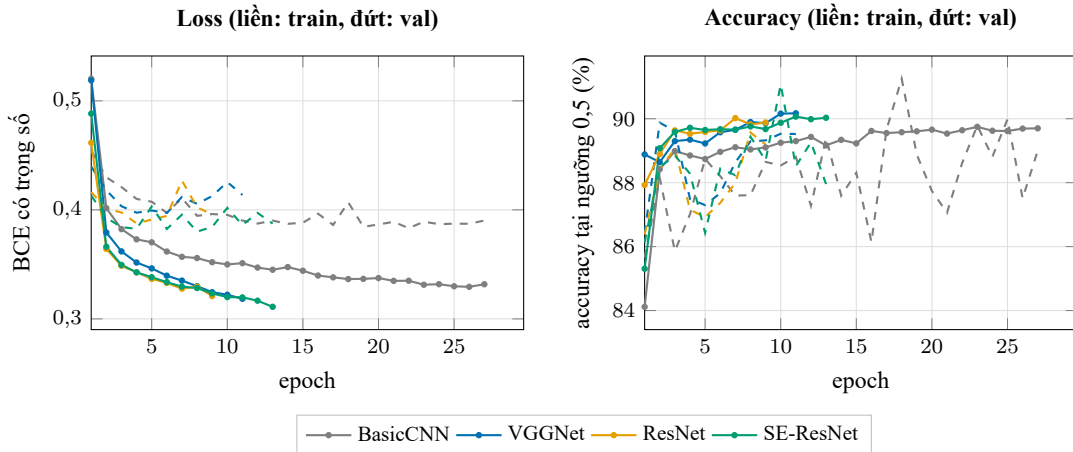
Chất lượng. Bốn mô hình gần như không phân biệt được. ROC-AUC chỉ dao động từ 0,9752 đến 0,9768, và PR-AUC từ 0,8779 đến 0,8816. Hai chỉ số này đo khả năng xếp hạng bệnh nhân theo nguy cơ và không phụ thuộc ngưỡng, nên có thể nói bốn mô hình xếp hạng tốt như nhau. F1 của SE-ResNet cao hơn (81,28 % so với 79,78 % của M0), nhưng Bảng 5.3 cho thấy phần chênh này đến từ *điểm cắt*: precision của nó cao nhất (95,34 %) còn recall thấp nhất (70,83 %). Nó không phải một mô hình tốt hơn, chỉ là một điểm khác trên cùng một đường ROC (Hình 5.3).



Hình 5.3. Đường ROC và Precision–Recall của bốn mô hình trên tập test Diabetes. Đường chấm trên hình PR là tỉ lệ dương, mốc của một bộ phân loại ngẫu nhiên

Vì sao kiến trúc không giúp gì ở đây. Cả ba cơ chế đều được thiết kế cho dữ liệu có cấu trúc không gian: bộ lọc 3×3 khai thác quan hệ giữa các điểm ảnh kề nhau, và một đặc trưng dịch đi vài ô vẫn là cùng một đặc trưng. Một hàng dữ liệu Diabetes chỉ có 15 cột, và hai cột kề nhau (ví dụ bmi và HbA1c_level) không có quan hệ lân cận nào. Chương 3 cũng cho thấy phần lớn ca dương tính đã được hai ngưỡng xét nghiệm tách ra. Phần khó còn lại nằm ở những bệnh nhân có chỉ số chồng lấn với người khỏe, và thêm tăng hay thêm cơ chế không tạo ra thông tin mới để phân biệt họ.

Hội tụ. Các mô hình có BN đạt val loss nhỏ nhất rất sớm (epoch 6 với M1, 4 với M2) rồi bị dừng sớm; M0 cần đến epoch 22 (Hình 5.4). Khoảng cách train–val ở epoch cuối nhỏ (0,6973 điểm với M0): với 67.302 dòng train và nhiều nhất 61.913 tham số, không mô hình nào quá khớp.

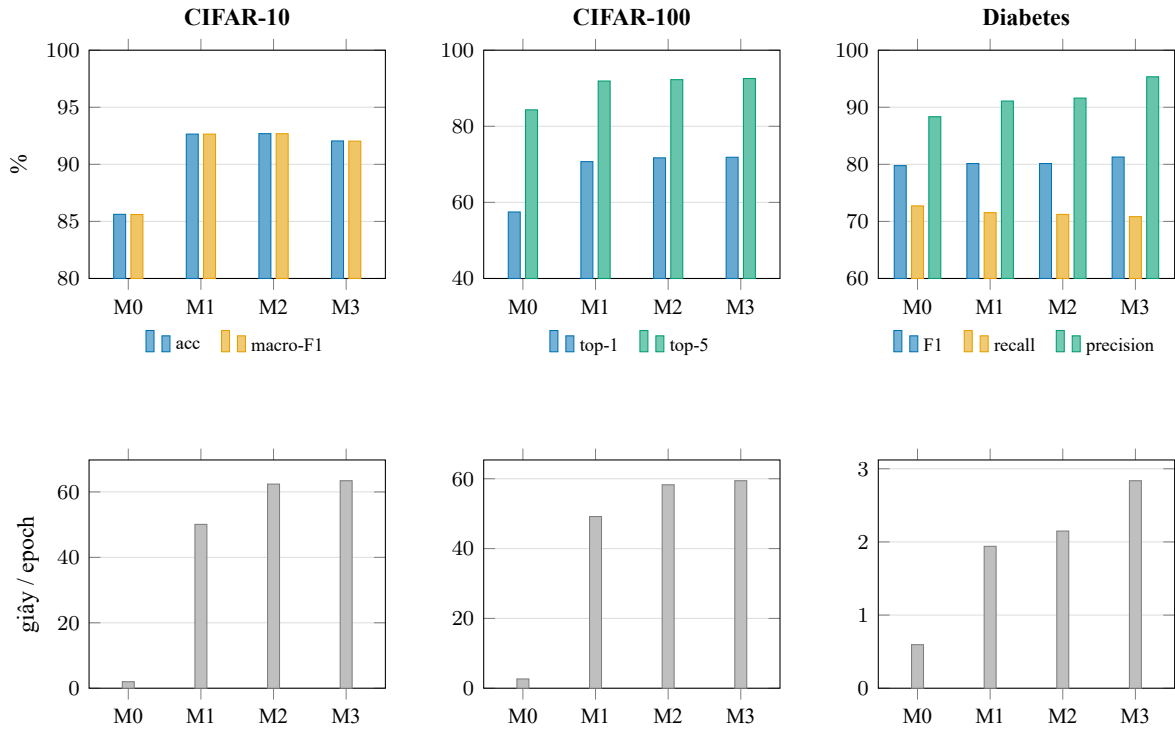


Hình 5.4. Đường học trên Diabetes; accuracy tính tại ngưỡng 0,5 của mô hình có pos_weight

Ma trận nhầm lẫn. Tại ngưỡng chọn trên validation, SE-ResNet bỏ sót 371 ca dương tính và báo nhầm 44 người khỏe; M0 bỏ sót 347 ca và báo nhầm 122 người. Nếu mô hình được dùng để sàng lọc, bỏ sót một ca bệnh đắt hơn nhiều so với một lần xét nghiệm thừa, và khi đó nên hạ ngưỡng để tăng recall thay vì chọn ngưỡng theo F1.

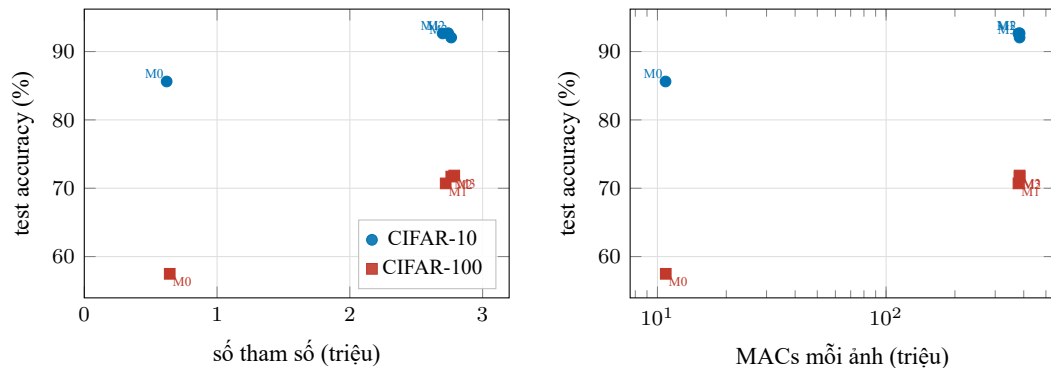
5.5 Chất lượng và chi phí

Hình 5.5 đặt chất lượng cạnh thời gian huấn luyện. Bước $M0 \rightarrow M1$ tăng số tham số từ 0,62 lên 2,70 triệu và số MACs mỗi ảnh từ 10,85 lên 379,26 triệu. Phần lớn MACs tăng thêm đến từ việc các tầng đầu của M1 chạy ở độ phân giải 32×32 với 64 kênh. Hai bước sau gần như miễn phí về tham số: đường tắt thêm 42.048 tham số (các conv 1×1 chiều) và SE thêm 22.456. Tuy vậy thời gian mỗi epoch vẫn tăng, từ 50,08 lên 62,4 giây, vì phép cộng đường tắt và SE là các thao tác đọc ghi bộ nhớ, không được tính vào MACs. Trên Diabetes, M0 huấn luyện một epoch trong 0,5955 giây, còn SE-ResNet cần 2,84 giây, cho cùng một chất lượng.



Hình 5.5. Chất lượng (hàng trên) và thời gian mỗi epoch (hàng dưới) của M0–M3 trên ba tập

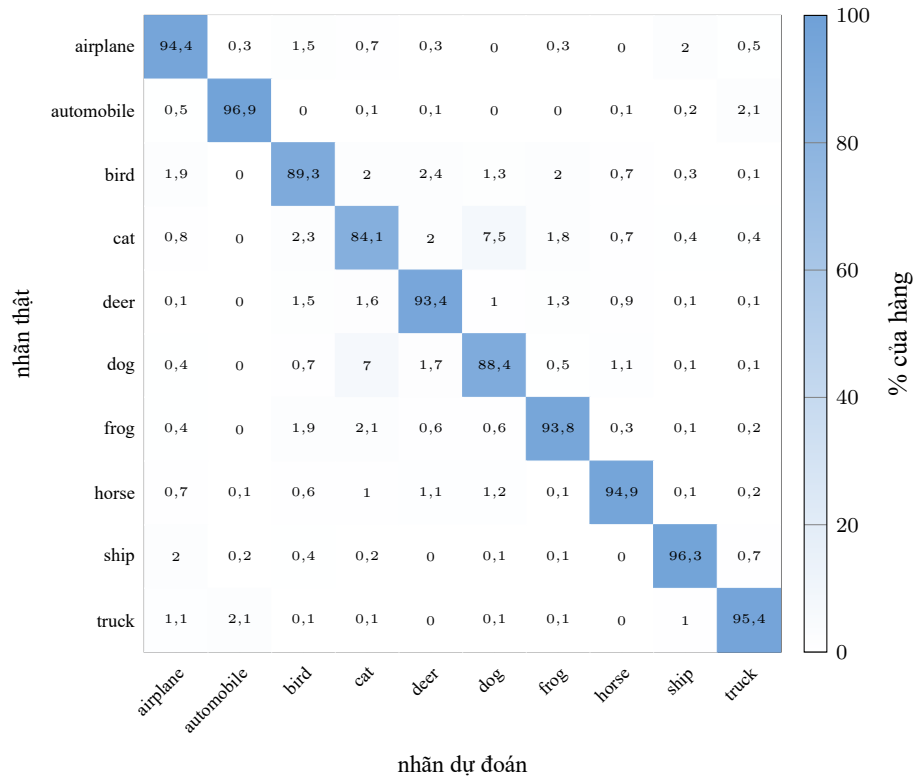
Hình 5.6 vẽ accuracy theo chi phí. Ba mô hình M1–M3 nằm chồng lên nhau ở cùng mức chi phí. Trên cả hai tập ảnh, đường nối M0 với cụm này dốc lên rõ rệt: số tham số tăng từ 0,62 lên 2,70 triệu đổi được 7,03 điểm trên CIFAR-10 và 13,23 điểm trên CIFAR-100.



Hình 5.6. Accuracy trên tập test theo số tham số (trái) và theo MACs mỗi ảnh, trục log (phải)

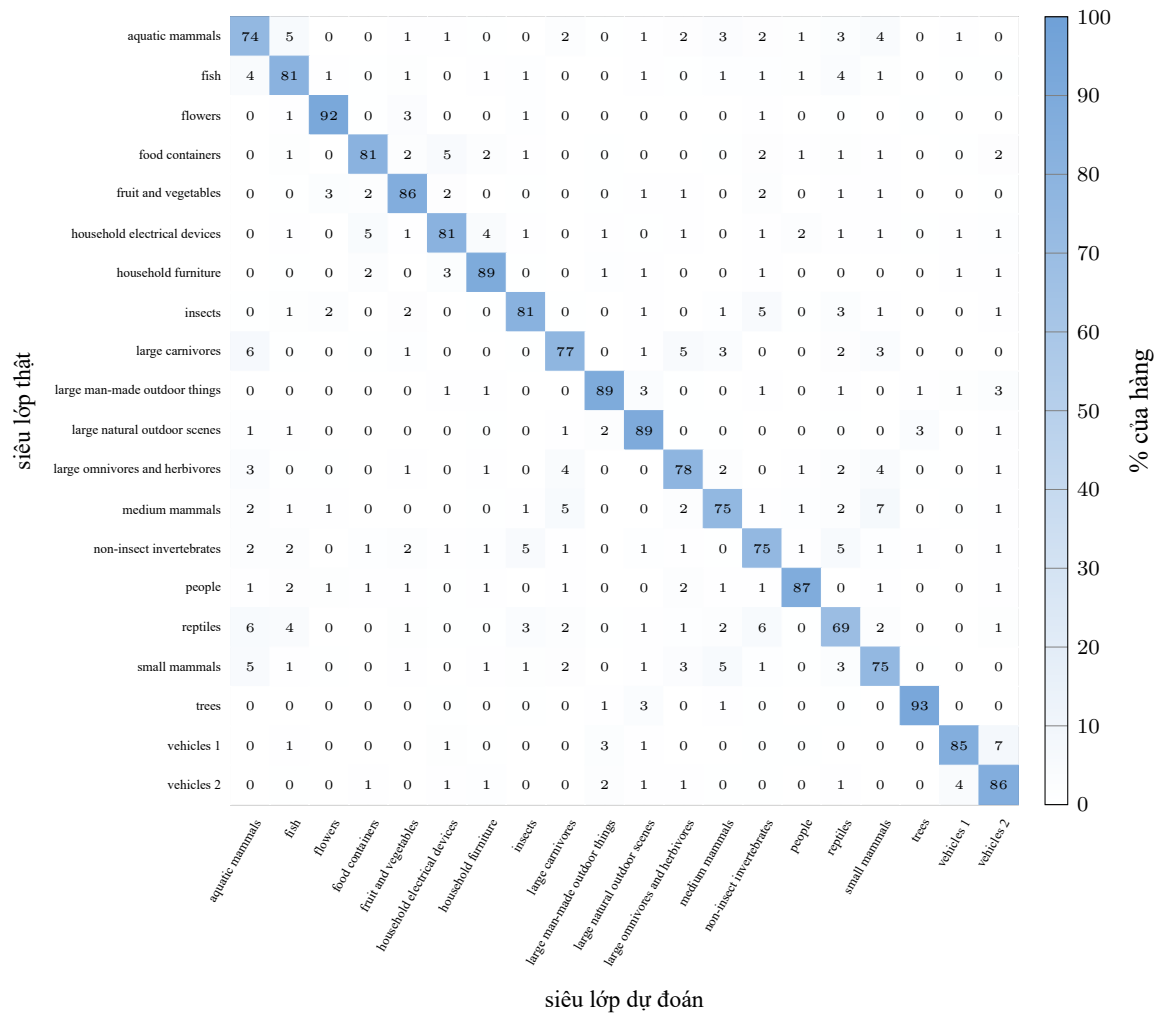
5.6 Phân tích lỗi

Hình 5.7 là ma trận nhầm lẫn của ResNet, mô hình tốt nhất trên CIFAR-10. Lớp khó nhất là cat (recall 84,1 %), lớp dễ nhất là automobile (96,9 %). Cặp nhầm nhiều nhất là cat bị đoán thành dog (7,5 % số ảnh cat). Hai lớp này đều là thú bốn chân có lông, và ở độ phân giải 32×32 chúng khác nhau chủ yếu ở hình dạng tai và mõm. Lỗi của mô hình đi theo sự giống nhau về hình ảnh chứ không rải ngẫu nhiên.



Hình 5.7. Ma trận nhầm lẫn CIFAR-10 của mô hình tốt nhất, % theo hàng

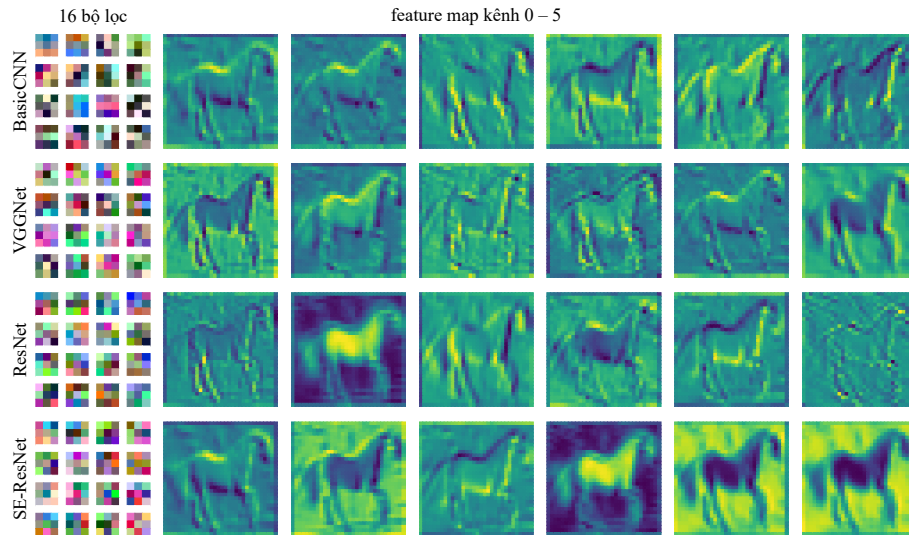
Với CIFAR-100, ma trận 100×100 quá dày để đọc, nên lớp thật và lớp dự đoán được gộp về 20 siêu lớp (Hình 5.8). Mô hình tốt nhất là SE-ResNet. Accuracy ở mức siêu lớp là 82,12 %, cao hơn hẳn top-1 ở mức lớp mịn (71,86 %). Như vậy rất nhiều lỗi là nhầm giữa hai lớp mịn *trong cùng một siêu lớp*: dự đoán sai lớp nhưng vẫn đúng nhóm. Cặp siêu lớp nhầm nhiều nhất là vehicles_1 sang vehicles_2 (7,4 %). Siêu lớp khó nhất là reptiles, dễ nhất là trees.



Hình 5.8. Ma trận nhầm lẫn CIFAR-100 gộp theo 20 siêu lớp, % theo hàng

5.7 Biểu diễn: bộ lọc lớp đầu và feature map

Hình 5.9 đặt cạnh nhau 16 bộ lọc $3 \times 3 \times 3$ đầu tiên của tầng tích chập đầu và sáu feature map mà tầng đó tạo ra từ cùng một ảnh test (lớp horse, ảnh ở Hình 4.2), cho bốn mô hình CIFAR-10. Mỗi bộ lọc được kéo giãn min-max về $[0, 255]$ để hiển thị như một ảnh màu nhỏ.



Hình 5.9. Bộ lọc lớp đầu và feature map của cùng một ảnh test qua bốn mô hình CIFAR-10

Ở tầng đầu, bốn mô hình học những thứ giống nhau: các bộ lọc là những mảng màu và những cặp sáng–tối theo một hướng, và feature map làm nổi đường viền con ngựa theo các hướng khác nhau. Một vài kênh của ResNet và SE-ResNet phản ứng với cả vùng thân ngựa thay vì chỉ với viền. Tầng đầu của bốn mô hình giống nhau như vậy cũng khớp với việc M1, M2, M3 cho accuracy gần như bằng nhau: khác biệt giữa chúng, nếu có, nằm ở các tầng sâu hơn.

Có một khác biệt đo được: độ lệch chuẩn của trọng số bộ lọc lớp đầu ở M0 là 0,3189, còn ở M1, M2, M3 chỉ từ 0,1076 đến 0,1516. Điều này có lý do: ở M1–M3, ngay sau mỗi tầng tích chập là BatchNorm, và BatchNorm chuẩn hoá lại đầu ra, nên nhân mọi trọng số của bộ lọc với một hằng số không làm đổi kết quả. Độ lớn của trọng số vì thế không còn ý nghĩa, và weight decay kéo nó nhỏ dần. M0 không có BatchNorm nên độ lớn trọng số quyết định trực tiếp độ lớn của feature map.

5.8 Tổng hợp: mỗi cơ chế mang lại bao nhiêu

Bảng 5.4 gom chênh lệch giữa hai mô hình liền kề trên cả ba tập, và Bảng 5.5 là toàn bộ 12 lượt chạy (3 tập $\times$ 4 mô hình).

Bảng 5.4. Chênh lệch chất lượng giữa hai mô hình liền kề

Bước	CIFAR-10 (acc)	CIFAR-100 (acc)	Diabetes (F1)	Tham số (CIFAR-10)
M0 $\rightarrow$ M1: sâu hơn + BN + GAP	+7,03	+13,23	+0,37	+2.078.592
M1 $\rightarrow$ M2: + đường tắt	+0,04	+1,00	+0,00	+42.048
M2 $\rightarrow$ M3: + SE	−0,64	+0,15	+1,14	+22.456

Đơn vị: điểm phần trăm. Chữ nghiêng: chênh lệch dưới 0,5 điểm, trong mức dao động của một lần chạy.

Bảng 5.5. Tổng hợp 12 lượt chạy trên tập test

Tập	Mô hình	Acc (%)	F1 (%)	Tham số	s/epoch	Epoch tốt/chạy
CIFAR-10	BasicCNN	85,62	85,60	620.362	2,00	30/30
	VGGNet	92,65	92,65	2.698.954	50,08	30/30
	ResNet	92,69	92,69	2.741.002	62,40	30/30
	SE-ResNet	92,05	92,03	2.763.458	63,43	28/30
CIFAR-100	BasicCNN	57,48	57,43	643.492	2,66	30/30
	VGGNet	70,71	70,79	2.722.084	49,18	29/30
	ResNet	71,71	71,71	2.764.132	58,28	29/30
	SE-ResNet	71,86	71,92	2.786.588	59,43	29/30
Diabetes	BasicCNN	96,75	79,78	12.065	0,60	22/27
	VGGNet	96,87	80,14	57.073	1,94	6/11
	ResNet	96,89	80,14	59.873	2,15	4/9
	SE-ResNet	97,12	81,28	61.913	2,84	8/13

F1 là macro-F1 với hai tập ảnh, F1 của lớp dương với Diabetes.

Ba điều rút ra từ hai bảng trên:

1. **Bước M0→M1 quyết định gần như toàn bộ chênh lệch trên ảnh:** 7,03 điểm trên CIFAR-10 và 13,23 điểm trên CIFAR-100. Bước này gộp ba thay đổi (sâu hơn, BatchNorm, GAP), và thí nghiệm không tách được phần đóng góp của từng thay đổi. Bằng chứng liên quan duy nhất là M0, mô hình không có BatchNorm, đã phân kỳ ở tốc độ học đỉnh khi chưa cắt gradient (Hình 4.3).
2. **Đường tắt chỉ có ích khi bài toán đủ khó:** không đổi gì trên CIFAR-10 (0,04 điểm), tăng 1 điểm trên CIFAR-100. Với mạng 12 tầng, lợi ích này nhỏ; nó được thiết kế cho mạng sâu hàng chục đến hàng trăm tầng.
3. **SE không cho lợi ích nhất quán:** -0,64 điểm trên CIFAR-10 (validation lại xếp nó đứng đầu), 0,15 điểm trên CIFAR-100. Trên Diabetes, F1 của nó cao hơn 1,14 điểm nhưng ROC-AUC không đổi, tức là khác biệt nằm ở ngưỡng chứ không ở mô hình.

Trên dữ liệu bảng, lựa chọn hợp lý là mô hình *re nhất*: M0 cho cùng khả năng xếp hạng với 12.065 tham số, so với 61.913 của M3. Trên ảnh, lựa chọn hợp lý là M1 hoặc M2: M2 tốt hơn một chút khi bài toán khó hơn, với chi phí tham số gần như bằng M1.

Chương 6

Kết luận

6.1 Trả lời năm mục của đề

1. **Khái niệm cơ bản qua hàm và hợp hàm** (Chương 1). Neuron, hàm kích hoạt, tích chập, pooling, softmax và cross-entropy được viết thành các hàm NumPy trong `a05/concepts.py`. CNN là hợp của các hàm đó, và lan truyền ngược là quy tắc đạo hàm của hàm hợp. 14/14 phép kiểm chứng khớp với PyTorch; gradient viết tay của tầng tích chập sai lệch so với sai phân hữu hạn chưa tới 10^{-6} .
2. **Các mô hình phát triển** (Chương 2). Chín mốc từ LeNet-5 đến ViT, mỗi mốc là một cơ chế viết được dưới dạng hàm: chồng conv 3×3 , nhánh song song, $F(x) + x$, nối mọi tầng, tách depthwise, nhân trọng số kênh, self-attention. Mỗi cơ chế có một lớp PyTorch trong `a05/blocks.py`, đã kiểm tra shape và số tham số.
3. **Dữ liệu** (Chương 3). CIFAR-10, CIFAR-100 (bài hiểu “MNIST-100” của đề là tập này) và Diabetes Prediction, kèm đường dẫn đã kiểm tra, thống kê mô tả, cách chia tập và dạng tensor. Diabetes bị bỏ 3.854 dòng trùng trước khi chia, và mọi phép biến đổi chỉ học trên nhánh train.
4. **Một CNN cơ bản và ba mô hình phát triển** (Chương 4). BasicCNN, VGGNet, ResNet, SE-ResNet, mỗi bước thêm đúng một cơ chế, có bản 2D cho ảnh và bản 1D cho Diabetes từ cùng một định nghĩa. Mười hai lượt huấn luyện ($3 \text{ tập} \times 4 \text{ mô hình}$) dùng chung một cấu hình trong mỗi tập.
5. **So sánh và đánh giá** (Chương 5). So theo chất lượng, chi phí, hội tụ, lỗi và biểu diễn. Kết quả tốt nhất: 92,69 % trên CIFAR-10 (ResNet), 71,86 % top-1 và 92,56 % top-5 trên CIFAR-100 (SE-ResNet), ROC-AUC 0,9768 trên Diabetes (BasicCNN).

6.2 Điều rút ra

Kết quả quan trọng nhất của bài không phải mô hình nào đứng đầu, mà là *một cơ chế chỉ có ích khi mô hình đang gặp đúng vấn đề mà cơ chế đó sinh ra để giải quyết*. BatchNorm được tạo ra để huấn luyện ổn định, và M0, mô hình duy nhất không có nó, là mô hình đã phân kỳ ở tốc độ học cao khi chưa cắt gradient. Đường tắt giải quyết bài toán tối ưu mạng sâu, nên nó không đổi gì khi M1 đã gần như thuộc lòng CIFAR-10, và chỉ thêm được 1 điểm trên CIFAR-100. Cả ba cơ chế đều khai thác cấu trúc không gian của ảnh, nên không giúp gì cho một hàng dữ liệu bảng không có cấu trúc đó. Đọc lịch sử CNN như một chuỗi “kiến trúc sau luôn tốt hơn kiến trúc trước” là đọc sai: mỗi kiến trúc tốt hơn *trong điều kiện* mà nó được thiết kế.

6.3 Hạn chế

- **Một hạt giống**. Mỗi mô hình chạy một lần. Chênh lệch dưới 0,5 điểm không được dùng để kết luận, và ngay cả chênh lệch 1 điểm của đường tắt trên CIFAR-100 cũng chỉ nên coi là dấu hiệu.
- **Bước M0→M1 gộp ba thay đổi**. Thí nghiệm không tách được phần đóng góp riêng của độ sâu, BatchNorm và GAP.
- **Mạng nông**. Với 12 tầng tích chập, thí nghiệm không kiểm tra được lợi ích của đường tắt ở độ sâu mà nó được thiết kế cho.
- **Thời gian đo trên GPU laptop bị hạ xung vì nhiệt**. Số giây mỗi epoch chỉ so sánh được giữa các mô hình với nhau.
- **“MNIST-100”** được hiểu là CIFAR-100. Nếu đề muốn một tập khác, Chương 3 và phần CIFAR-100 của Chương 5 cần làm lại với tập đó; mã nguồn không phải đổi gì ngoài hàm nạp dữ liệu.

Tài liệu tham khảo

- [1] Yann LeCun, Léon Bottou, Yoshua Bengio **and** Patrick Haffner. “Gradient-Based Learning Applied to Document Recognition”. *in**Proceedings of the IEEE*: 86.11 (1998), **pages** 2278–2324.
- [2] Alex Krizhevsky, Ilya Sutskever **and** Geoffrey E. Hinton. “ImageNet Classification with Deep Convolutional Neural Networks”. *in**Advances in Neural Information Processing Systems* 25: 2012, **pages** 1097–1105.
- [3] Karen Simonyan **and** Andrew Zisserman. “Very Deep Convolutional Networks for Large-Scale Image Recognition”. *in**International Conference on Learning Representations (ICLR)*: 2015.
- [4] Christian Szegedy **and** others. “Going Deeper with Convolutions”. *in**IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*: 2015, **pages** 1–9.
- [5] Kaiming He, Xiangyu Zhang, Shaoqing Ren **and** Jian Sun. “Deep Residual Learning for Image Recognition”. *in**IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*: 2016, **pages** 770–778.
- [6] Gao Huang, Zhuang Liu, Laurens van der Maaten **and** Kilian Q. Weinberger. “Densely Connected Convolutional Networks”. *in**IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*: 2017, **pages** 4700–4708.
- [7] Andrew G. Howard **and** others. *MobileNets: Efficient Convolutional Neural Networks for Mobile Vision Applications*. arXiv:1704.04861. 2017.
- [8] Jie Hu, Li Shen **and** Gang Sun. “Squeeze-and-Excitation Networks”. *in**IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*: 2018, **pages** 7132–7141.
- [9] Sanghyun Woo, Jongchan Park, Joon-Young Lee **and** In So Kweon. “CBAM: Convolutional Block Attention Module”. *in**European Conference on Computer Vision (ECCV)*: 2018, **pages** 3–19.
- [10] Alexey Dosovitskiy, Lucas Beyer, Alexander Kolesnikov, Dirk Weissenborn, Xiaohua Zhai, Thomas Unterthiner **and** others. “An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale”. *in**International Conference on Learning Representations (ICLR)*: 2021.
- [11] Alex Krizhevsky. *Learning Multiple Layers of Features from Tiny Images*. techreport. Nguồn của CIFAR-10 và CIFAR-100, truy cập 2026-09-25. University of Toronto, 2009. URL: <https://www.cs.toronto.edu/~kriz/cifar.html>.
- [12] Mohammed Mustafa. *Diabetes Prediction Dataset*. Kaggle. 100.000 hồ sơ bệnh nhân, truy cập 2026-09-25. 2023. URL: <https://www.kaggle.com/datasets/iammustafatz/diabetes-prediction-dataset>.
- [13] Adam Paszke, Sam Gross, Francisco Massa **and** others. “PyTorch: An Imperative Style, High-Performance Deep Learning Library”. *in**Advances in Neural Information Processing Systems* 32: 2019, **pages** 8024–8035.
- [14] Sergey Ioffe **and** Christian Szegedy. “Batch Normalization: Accelerating Deep Network Training by Reducing Internal Covariate Shift”. *in**International Conference on Machine Learning (ICML)*: 2015, **pages** 448–456.
- [15] Leslie N. Smith **and** Nicholay Topin. “Super-Convergence: Very Fast Training of Neural Networks Using Large Learning Rates”. *in**Artificial Intelligence and Machine Learning for Multi-Domain Operations Applications: volume* 11006. Proceedings of SPIE. 2019.

Phụ lục A

Tái lập kết quả

A.1 Môi trường

Bảng A.1. Môi trường chạy thực nghiệm (ghi tự động bởi notebook 06)

Hệ điều hành	Windows-11
Python	3.13.5
PyTorch	2.13.0+cu126 (CUDA 12.6, cuDNN 91002)
NumPy, pandas, scikit-learn	2.5.1, 3.0.5, 1.9.0
GPU	NVIDIA GeForce RTX 4060 Laptop GPU

A.2 Các bước

Toàn bộ mã nguồn nằm trong `src/Assignment 05/`. Chạy lại từ đầu gồm bốn bước:

```
cd "src/Assignment 05"
pip install torch==2.13.0 --index-url https://download.pytorch.org/whl/cu126
pip install -r requirements.txt
python -m ipykernel install --user --name a05

# 1. Dữ liệu ảnh (CSV Diabetes đã có sẵn trong data/diabetes/)
curl -L -o data/cifar10/cifar-10-python.tar.gz https://www.cs.toronto.edu/~kriz/cifar-10-python.tar.gz
curl -L -o data/cifar100/cifar-100-python.tar.gz https://www.cs.toronto.edu/~kriz/cifar-100-python.tar.gz

# 2. Bấy notebook theo thứ tự (06 đọc kết quả của 03, 04, 05)
cd notebooks
for nb in 00 01 02 03 04 05 06; do
    python -m jupyter nbconvert --to notebook --execute --inplace \
        --ExecutePreprocessor.kernel_name=a05 --ExecutePreprocessor.timeout=-1 ${nb}*.ipynb
done

# 3. Báo cáo: chép dữ liệu hình, trích mã, sinh macro số liệu, gọi latexmk -LuaLatex
python "../../Report/Assignment 05/build.py"
```

A.3 Từ mã nguồn đến báo cáo

Báo cáo này không chứa con số, đoạn mã hay hình vẽ nào được chép tay:

- Số liệu.** Mỗi notebook ghi kết quả vào `outputs/metrics/*.json`. `build.py` biến mỗi khoá JSON thành một macro, ví dụ khoá `cifar10` → `models` → `resnet` → `acc` thành `\SL{cifar10/models/resnet/acc}`, in theo định dạng số tiếng Việt. Bảng kết quả cũng được sinh từ các tệp JSON này.
- Mã nguồn.** Mỗi lệnh `\maNgon{<module>__<ten>}{...}` trong chương khiến `build.py` dùng `ast` cắt đúng thân hàm hoặc lớp `<ten>` trong `a05/<module>.py`. Sửa mã nguồn thì báo cáo đổi theo ở lần dựng sau.
- Hình vẽ.** Mọi hình là TikZ/pgfplots, đọc từ `outputs/figdata/*.dat`. Ảnh mẫu, bộ lọc và feature map được ghi thành bảng giá trị điểm ảnh rồi vẽ bằng `matrix plot*`. Mỗi hình được biên dịch riêng một lần (TikZ externalize); `build.py` bấm mã hình cùng dữ liệu nó đọc để biết hình nào cần vẽ lại.